SIMULASI ALIRAN DUA FASE PIPA KALOR NATRIUM UNTUK REAKTOR ANTARIKSA PADA MIKROGRAVITASI

SKRIPSI

untuk memenuhi sebagian persyaratan
untuk memperoleh derajat Sarjana
Program Studi Teknik Nuklir

Diajukan oleh

MUHAMMAD NAUFAL TAMIMI
20/456315/TK/50445

Kepada

DEPARTEMEN TEKNIK NUKLIR DAN TEKNIK FISIKA
FAKULTAS TEKNIK
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini:

Nama	:	Muhammad Naufal Tamimi
NIM	:	20/456315/TK/50445
Tahun terdaftar	:	[tahun terdaftar]
Program Studi	:	Teknik Nuklir
Fakultas	:	Teknik

menyatakan bahwa dokumen ilmiah skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, [tanggal-bulan-tahun]
Materai dan tanda tangan

Muhammad Naufal Tamimi
NIM. 20/456315/TK/50445

HALAMAN PENGESAHAN

SKRIPSI

SIMULASI ALIRAN DUA FASE PIPA KALOR NATRIUM UNTUK REAKTOR ANTARIKSA PADA MIKROGRAVITASI

Nama Mahasiswa : Muhammad Naufal Tamimi

Nomor Mahasiswa : 20/456315/TK/50445

Pembimbing Utama : [nama dan gelar]

Pembimbing Pendamping : [nama dan gelar]

Skripsi ini telah dipertahankan di depan Tim Penguji pada tanggal [tanggal ujian].

Ketua Sidang : [nama dan gelar]

Penguji Utama : [nama dan gelar]

Anggota Penguji : [nama dan gelar]

Skripsi ini telah diterima dan dinyatakan memenuhi syarat kelulusan pada tanggal [tanggal kelulusan].

Ketua Departemen Teknik Nuklir dan Teknik Fisika
Fakultas Teknik UGM

[nama dan gelar]
NIP. [nomor induk pegawai]

HALAMAN PERSEMBAHAN

Karya ini dipersembahkan kepada keluarga dan para pendidik yang telah memberikan dukungan selama proses pembelajaran.

MOTTO

Model yang baik bukan hanya menghasilkan angka, tetapi juga menjelaskan asumsi dan batas keberlakuannya.

KATA PENGANTAR

Skripsi ini menyajikan pengembangan simulator aliran dua fase pada pipa kalor natrium untuk kajian awal sistem pembuangan kalor reaktor antariksa pada kondisi mikrogravitasi. Pembahasan diarahkan pada hubungan antara transportasi kalor, aliran uap, aliran balik cairan di dalam sumbu kapiler, dan batas operasi pipa kalor. Program disusun secara modular agar persamaan, parameter masukan, dan hasil dapat diperiksa serta direproduksi.

Penulis menyampaikan terima kasih kepada pembimbing, dosen, tenaga kependidikan, keluarga, dan rekan-rekan yang telah memberikan dukungan selama penyusunan tugas akhir. Saran dan koreksi diperlukan terutama untuk pengembangan model transien, perbaikan sifat fluida, dan validasi eksperimental.

Yogyakarta, [tanggal] Oktober 2026

Penulis

DAFTAR ISI

PERNYATAAN BEBAS PLAGIASI i

HALAMAN PENGESAHAN ii

HALAMAN PERSEMBAHAN iii

MOTTO iv

KATA PENGANTAR v

DAFTAR LAMBANG DAN SINGKATAN x

INTISARI xii

ABSTRACT xiii

I PENDAHULUAN 1

I.1. Latar Belakang 1

I.2. Perumusan Masalah 2

I.2.1. Batasan Masalah 2

I.3. Tujuan Penelitian 3

I.4. Manfaat Penelitian 3

II TINJAUAN PUSTAKA 4

II.1. Perkembangan Pipa Kalor 4

II.2. Pipa Kalor untuk Reaktor Antariksa 4

II.3. Pemodelan Aliran Dua Fase Pipa Kalor 5

II.4. Posisi dan Kebaruan Penelitian 6

III DASAR TEORI 7

III.1. Deskripsi Model dan Volume Kontrol 7

III.2. Neraca Energi dan Massa 7

III.3. Sifat Natrium 8

III.4. Aliran Uap 9

III.5. Aliran Balik Cairan dan Tekanan Kapiler 10

III.6. Batas Transportasi Kalor 10

III.7. Metode Numerik 11

III.8. Hipotesis 11

IV PELAKSANAAN PENELITIAN **13**
IV.1. Alat dan Bahan Penelitian 13
IV.2. Tata Laksana Penelitian 14
IV.3. Rencana Analisis Hasil Penelitian 15

V HASIL DAN PEMBAHASAN **17**
V.1. Hasil Kasus Acuan 17
V.2. Profil Aksial 18
V.3. Batas Operasi dan Pengaruh Temperatur 18
V.4. Pengaruh Beban Kalor 21
V.5. Pengaruh Percepatan Efektif 21
V.6. Pengaruh Jari-Jari Pori 22
V.7. Konvergensi Grid dan Verifikasi 23
V.8. Pembahasan Keterbatasan Model 24

VI KESIMPULAN DAN SARAN **25**
VI.1. Kesimpulan 25
VI.2. Saran 25

DAFTAR PUSTAKA **27**

A LISTING PROGRAM SIMULATOR **29**

B PENGUJIAN DAN PANDUAN PENGGUNAAN **49**
B.1. Listing Pengujian 49
B.2. Menjalankan Program 52
B.3. Interpretasi Status Program 52

DAFTAR TABEL

IV.1 Parameter geometri, sumbu, dan kondisi batas kasus acuan. 13
IV.2 Rancangan verifikasi perangkat lunak. 15

V.1 Sifat natrium pada temperatur operasi 1000 K. 17
V.2 Ringkasan hasil kasus acuan 800 W pada mikrogravitasi. 17
V.3 Peta batas transportasi kalor pada temperatur tetap. 20
V.4 Studi beban dengan temperatur sink dan resistansi kondenser tetap. 21
V.5 Pengaruh percepatan efektif pada batas kapiler pada 1000 K. 22
V.6 Pengaruh jari-jari pori dengan permeabilitas dipertahankan tetap. . 23
V.7 Konvergensi jumlah rugi tekanan uap dan cairan pada kasus acuan. 23

DAFTAR GAMBAR

III.1 Skema volume kontrol dan jalur aliran uap–cair yang digunakan dalam model. Garis putus-putus menunjukkan batas evaporator, adiabatik, dan kondenser. 7

V.1 Profil kalor aksial, kecepatan uap, dan rugi tekanan kumulatif untuk kasus acuan. Posisi $x = 0$ adalah ujung evaporator; garis batas berada pada 0,25 m dan 0,40 m. 19

V.2 Peta batas operasi terhadap temperatur. Panel atas menampilkan batas kapiler, sonik, entrainment, dan beban acuan; panel bawah menampilkan batas pendidihan dalam skala logaritmik. 20

V.3 Perbandingan batas kapiler pada mikrogravitasi, beberapa akselerasi, dan kondisi Bumi dengan orientasi merugikan. 22

DAFTAR LAMBANG DAN SINGKATAN

Lambang Romawi

Lambang	Kuantitas	Satuan
A	Luas penampang	m²
D_v	Diameter ruang uap	m
f	Faktor gesek Darcy	–
g_{eff}	Percepatan efektif	m/s²
h_{fg}	Kalor laten penguapan	J/kg
K	Permeabilitas sumbu kapiler	m²
L	Panjang pipa kalor	m
$\dot{m}$	Laju aliran massa	kg/s
P	Tekanan	Pa
Q	Laju transportasi kalor	W
r_c	Jari-jari pori efektif	m
R_c	Resistansi termal kondenser	K/W
T	Temperatur	K
u	Kecepatan uap	m/s
x	Koordinat aksial	m

Lambang Yunani

Lambang	Kuantitas	Satuan
θ	Sudut kontak atau kemiringan, sesuai konteks	°
μ	Viskositas dinamis	Pa·s
ρ	Densitas	kg/m³
σ	Tegangan permukaan	N/m

Subskrip dan Singkatan

Lambang	Deskripsi
c	Kondenser atau kapiler, sesuai konteks
e	Evaporator
l	Fasa cair
v	Fasa uap

Lambang	Deskripsi
CFD	*Computational Fluid Dynamics*
HPCR	*Heat Pipe Cooled Reactor*
NIM	Nomor Induk Mahasiswa
UGM	Universitas Gadjah Mada

INTISARI

Pipa kalor berpotensi digunakan pada reaktor antariksa karena mampu memindahkan kalor tanpa pompa. Pada mikrogravitasi, penggerak utama sirkulasi adalah tekanan kapiler sehingga neraca tekanan uap, cairan, dan gaya kapiler perlu dianalisis. Penelitian ini bertujuan mengembangkan simulator Python yang transparan untuk menghitung aliran dua fase tunak satu dimensi dan batas transportasi kalor pipa kalor natrium.

Model menggabungkan neraca energi, aliran uap aksial, aliran Darcy pada sumbu kapiler, dan tekanan kapiler. Sifat natrium dihitung dengan korelasi Fink dan Leibowitz. Kasus acuan menggunakan panjang 1,00 m, diameter ruang uap 12 mm, beban 800 W, serta temperatur operasi 1000 K. Penyelesaian numerik menggunakan diskretisasi aksial, integrasi trapesium, dan biseksi.

Hasil kasus acuan memberikan batas kapiler 2064,88 W, rugi tekanan uap 103,02 Pa, rugi tekanan cairan 1296,43 Pa, dan margin kapiler 2212,68 Pa. Batas sonik, entrainment, dan pendidihan masing-masing adalah 7155,53 W, 3212,10 W, dan 24.232,80 W. Kajian menunjukkan perpindahan batas pengatur dari sonik pada temperatur rendah ke kapiler pada temperatur lebih tinggi. Model terverifikasi melalui sepuluh pengujian, tetapi masih memerlukan validasi eksperimen.

Kata kunci: pipa kalor, natrium, aliran dua fase, mikrogravitasi, reaktor antariksa.

ABSTRACT

Heat pipes can support space-reactor heat removal without mechanical pumps. Under microgravity, circulation is driven primarily by capillary pressure; therefore, the vapor, liquid, and capillary pressure balance must be evaluated. This thesis develops a transparent Python simulator for steady one-dimensional two-phase flow and heat-transport limits in a sodium heat pipe.

The model couples an energy balance, axial vapor flow, Darcy liquid return through the wick, and capillary pressure. Sodium properties are evaluated using Fink and Leibowitz correlations. The reference case has a length of 1.00 m, a 12 mm vapor-core diameter, an 800 W heat load, and a 1000 K operating temperature. Axial discretization, trapezoidal integration, and bisection are used.

The reference calculation gives a capillary limit of 2064.88 W, vapor and liquid pressure drops of 103.02 Pa and 1296.43 Pa, and a capillary margin of 2212.68 Pa. The sonic, entrainment, and boiling screening limits are 7155.53 W, 3212.10 W, and 24,232.80 W, respectively. The operating map shifts from sonic limitation at low temperature to capillary limitation at higher temperature. Ten verification tests pass; experimental validation remains necessary.

Keywords: heat pipe, sodium, two-phase flow, microgravity, space reactor.

BAB I
PENDAHULUAN

I.1. Latar Belakang

Reaktor antariksa memerlukan sistem pembuangan kalor yang andal, ringkas, dan mampu bekerja tanpa bergantung pada orientasi gravitasi. Pipa kalor memenuhi sebagian kebutuhan tersebut karena memanfaatkan penguapan dan kondensasi fluida kerja untuk memindahkan kalor dari evaporator ke kondenser. Tidak seperti loop berpompa, pipa kalor konvensional menggunakan struktur berpori untuk mengembalikan cairan secara pasif. Konsep ini telah berkembang dari penemuan awal menjadi salah satu pilihan arsitektur pembuangan kalor pada reaktor berpendingin pipa kalor [1, 8-10].

Fluida alkali seperti natrium sesuai untuk pipa kalor temperatur tinggi karena memiliki kalor laten besar dan sifat transportasi yang mendukung operasi pada rentang temperatur reaktor. Natrium berubah menjadi uap pada evaporator, mengalir melalui ruang uap, berkondensasi pada kondenser, kemudian kembali sebagai cairan melalui sumbu kapiler. Proses tersebut merupakan aliran dua fase yang saling terkopel: laju penguapan menentukan laju aliran massa uap, sedangkan laju kondensasi menentukan jumlah cairan yang harus dikembalikan. Ketidakseimbangan antara tekanan pendorong kapiler dan rugi tekanan dapat menyebabkan kekurangan cairan serta pengeringan evaporator [3-6].

Mikrogravitasi menghilangkan komponen tekanan hidrostatik yang biasa digunakan untuk membantu atau menghambat pengembalian cairan di Bumi. Akibatnya, desain tidak dapat mengandalkan gaya berat untuk mengatur distribusi cairan. Pada sisi lain, hilangnya tekanan hidrostatik juga dapat menguntungkan operasi pada orientasi yang di Bumi bersifat merugikan. Kajian parameter percepatan efektif diperlukan untuk membedakan pengaruh kapiler, viskositas, dan orientasi secara jelas.

Pemodelan rinci berbasis dinamika fluida komputasional dapat menyelesaikan banyak fenomena, tetapi membutuhkan data material, kondisi batas, dan biaya komputasi yang besar. Untuk tugas akhir dan studi awal parameter, model tereduksi yang dapat dibaca, diuji, dan direproduksi memiliki nilai tersendiri. Model demikian tidak menggantikan CFD atau eksperimen, melainkan menyediakan hubungan fisis yang jelas antara masukan dan keluaran. Oleh karena itu, penelitian ini mengembangkan simulator Python untuk aliran dua fase tunak satu dimensi pada pipa kalor natrium, dengan penekanan pada neraca tekanan kapiler dan batas operasi dalam konteks mikrogravitasi.

I.2. Perumusan Masalah

Permasalahan utama yang dibahas adalah bagaimana menentukan kemampuan transportasi kalor pipa kalor natrium tanpa menggunakan model dua fase yang terlalu kompleks, tetapi tetap mempertahankan hubungan fisis utama antara aliran uap dan aliran balik cairan. Pertanyaan penelitian dirumuskan sebagai berikut.

1. Bagaimana merumuskan dan mengimplementasikan model tunak satu dimensi yang mengkopelkan neraca energi, aliran uap, aliran cairan Darcy, dan tekanan kapiler?
2. Berapa rugi tekanan, kecepatan uap, bilangan Reynolds, bilangan Mach, dan batas transportasi kalor pada kasus acuan?
3. Bagaimana temperatur operasi, ukuran pori, dan percepatan efektif memengaruhi batas operasi?
4. Apakah implementasi numerik memenuhi pengujian konsistensi sifat fluida, neraca profil energi, kondisi batas kapiler, dan konvergensi grid?

Pendekatan yang dipilih adalah model jaringan tereduksi dengan sifat fluida jenuh pada satu temperatur operasi. Panjang pipa dibagi menjadi evaporator, bagian adiabatik, dan kondenser. Rugi tekanan dihitung melalui integrasi aksial, sedangkan batas kapiler dicari dengan metode biseksi. Batas sonik, entrainment, dan pendidihan digunakan sebagai penyaringan tambahan.

I.2.1. Batasan Masalah

Penelitian dibatasi oleh asumsi dan ruang lingkup berikut.

1. Analisis hanya untuk satu pipa kalor natrium dan tidak menghitung neutronik, perpindahan kalor bahan bakar, atau konversi daya reaktor.
2. Model bersifat tunak, satu dimensi aksial, dan menggunakan temperatur uap jenuh seragam.
3. Evaporator dan kondenser menerima atau melepas fluks kalor merata; bagian adiabatik tidak kehilangan kalor.
4. Sumbu kapiler homogen, jenuh sempurna, memiliki permeabilitas dan jari-jari pori efektif konstan, serta membasahi sempurna pada kasus acuan.
5. Uap dimodelkan sebagai gas ideal dan cairan di dalam sumbu mengikuti hukum Darcy.
6. Rugi tekanan uap menggunakan korelasi faktor gesek aliran internal; kompresibilitas hanya diperiksa melalui bilangan Mach, bukan diselesaikan dengan persamaan aliran kompresibel penuh.
7. Batas sonik, entrainment, dan pendidihan merupakan model penyaringan rekayasa. Batas viskosit saat start-up beku, gas takterkondensasi, dan perubahan

geometri tidak dimodelkan.

8. Kasus acuan bersifat representatif untuk studi metode dan bukan spesifikasi desain reaktor tertentu.
9. Verifikasi dilakukan terhadap sifat referensi dan konsistensi numerik; validasi terhadap data eksperimen belum dilakukan.

I.3. Tujuan Penelitian

Tujuan penelitian ini adalah sebagai berikut.

1. Mengembangkan simulator Python yang modular dan terdokumentasi untuk analisis aliran dua fase pipa kalor natrium.
2. Menghitung profil aksial, neraca tekanan, dan batas kapiler, sonik, entrainment, serta pendidihan pada kasus acuan mikrogravitasi.
3. Menganalisis pengaruh temperatur, beban kalor, jari-jari pori, dan percepatan efektif terhadap kemampuan transportasi kalor.
4. Memverifikasi implementasi melalui pengujian otomatis dan studi konvergensi grid.
5. Menyajikan kode, data, dan dokumen skripsi dalam struktur Quarto yang dapat direproduksi.

I.4. Manfaat Penelitian

Penelitian ini diharapkan memberikan model pembelajaran yang menghubungkan persamaan pipa kalor dengan implementasi komputasi. Bagi penelitian lanjutan, kode dapat digunakan sebagai titik awal untuk studi sensitivitas sebelum pemodelan berdimensi lebih tinggi. Bagi pembaca, pemisahan yang tegas antara asumsi, verifikasi, dan validasi membantu mencegah penafsiran berlebihan terhadap hasil numerik. Dokumen Quarto juga memungkinkan persamaan, gambar, tabel, dan listing program dikelola dalam satu proyek yang dapat diperbarui.

BAB II
TINJAUAN PUSTAKA

II.1. Perkembangan Pipa Kalor

Prinsip perangkat berkonduktansi termal tinggi yang menggunakan perubahan fasa diperkenalkan melalui karya awal Gaugler dan kemudian Grover. Publikasi Grover, Cotter, dan Erickson pada 1964 memperlihatkan struktur berkonduktansi termal sangat tinggi yang menjadi dasar pipa kalor modern [1]. Cotter selanjutnya merumuskan teori awal mengenai hubungan kapiler, aliran cairan, dan aliran uap [2]. Buku acuan Chi serta Faghri kemudian menyusun persamaan batas transportasi dan metode analisis yang banyak digunakan dalam perancangan pipa kalor [3, 4].

Pipa kalor terdiri atas evaporator, bagian adiabatik, dan kondenser. Fluida menerima kalor laten pada evaporator dan melepaskannya pada kondenser. Perbedaan tekanan kapiler pada antarmuka cair–uap memungkinkan cairan kembali melalui struktur pori. Kemampuan operasi tanpa pompa ini menjadikan pipa kalor berbeda dari loop dua fase konvensional. Namun, kapasitasnya tidak tak terbatas. Tekanan kapiler harus mengatasi rugi tekanan cairan, rugi tekanan uap, dan tekanan akibat percepatan eksternal [4-6].

Struktur sumbu dapat berupa anyaman kawat, serbuk tersinter, atau alur. Setiap struktur memiliki hubungan berbeda antara jari-jari pori, permeabilitas, porositas, dan konduktivitas efektif. Pori kecil meningkatkan tekanan kapiler, tetapi sering diikuti permeabilitas rendah. Oleh sebab itu, perubahan jari-jari pori tidak selalu dapat ditafsirkan terpisah dari permeabilitas pada desain nyata. Model penelitian ini memperlakukan keduanya sebagai parameter independen agar pengaruh masing-masing dapat dipelajari secara terkontrol.

II.2. Pipa Kalor untuk Reaktor Antariksa

Reaktor berpendingin pipa kalor menggunakan sejumlah pipa yang tertanam atau terhubung secara termal dengan teras. Arsitektur ini dapat mengurangi kebutuhan pompa dan pipa utama, serta memberikan redundansi apabila kegagalan satu pipa tidak langsung menghentikan seluruh sistem. Yan dan rekan-rekannya membahas perkembangan mikroreaktor berpendingin pipa kalor beserta tantangan kegagalan berantai, integritas struktur, dan kualifikasi [8]. Tinjauan Wang, Ren, dan Li menunjukkan bahwa berbagai konsep menggunakan natrium, kalium, atau litium sesuai temperatur operasi dan arsitektur sistem [9].

Dalam konteks antariksa, kondisi mikrogravitasi menghilangkan gaya apung dan tekanan hidrostatik berskala besar. Pipa kalor bersumbu tetap dapat bekerja

karena pengembalian cairan didorong oleh kapilaritas. Sebaliknya, termosifon tanpa sumbu sangat bergantung pada gravitasi sehingga tidak menjadi pilihan utama dalam model ini. Keunggulan pasif tersebut tetap disertai tantangan start-up dari kondisi beku, keberadaan gas takterkondensasi, kompatibilitas material, dan pembuangan kalor dari kondenser ke radiator [8-10].

Pemilihan natrium dalam penelitian ini didasarkan pada penggunaannya pada pipa kalor temperatur tinggi dan tersedianya korelasi sifat yang dievaluasi secara termodinamika. Fink dan Leibowitz menyajikan persamaan tekanan uap, entalpi penguapan, densitas, tegangan permukaan, viskositas, dan konduktivitas natrium [7]. Ketersediaan sumber tersebut memungkinkan implementasi diperiksa terhadap nilai tabel, bukan hanya terhadap keluaran program itu sendiri.

II.3. Pemodelan Aliran Dua Fase Pipa Kalor

Model pipa kalor dapat dikelompokkan menjadi model analitik, model jaringan, dan model CFD. Model analitik menggunakan penyederhanaan untuk memperoleh batas transportasi dan neraca tekanan. Model jaringan membagi pipa menjadi elemen termal atau hidraulik yang saling berhubungan. Model CFD menyelesaikan medan kecepatan, tekanan, temperatur, dan dalam beberapa pendekatan antarmuka fasa. Ketiga tingkat model memiliki tujuan berbeda; model yang lebih rinci tidak selalu diperlukan untuk penyaringan parameter awal [4, 9].

HTPIPE merupakan contoh program analisis pipa kalor tunak yang dikembangkan untuk membantu perhitungan kinerja dan batas operasi [11]. Perkembangan berikutnya mencakup model sistem reaktor seperti HEART yang menghubungkan analisis termal dan elektrik [12]. Pada tingkat eksperimen, Wahlquist dan rekan-rekannya menekankan pentingnya kualitas serta cakupan data untuk validasi pipa kalor pada aplikasi energi nuklir [13]. Literatur tersebut memperlihatkan bahwa kode tidak dapat dipisahkan dari pertanyaan mengenai kondisi batas, data material, dan validasi.

Persamaan kapiler umum digunakan untuk menentukan apakah tekanan pendorong mencukupi. Batas lain muncul ketika kecepatan uap mendekati kondisi sonik, tetes cairan terbawa aliran uap, atau pendidihan di dalam sumbu menghalangi pengembalian cairan. Pada temperatur rendah, densitas uap yang kecil dapat menyebabkan batas sonik dan viskositas menjadi penting. Pada temperatur lebih tinggi, batas kapiler atau entrainment dapat menjadi lebih menonjol. Urutan batas bergantung pada geometri dan kondisi batas sehingga harus dihitung, bukan ditetapkan sebelumnya [3-6].

II.4. Posisi dan Kebaruan Penelitian

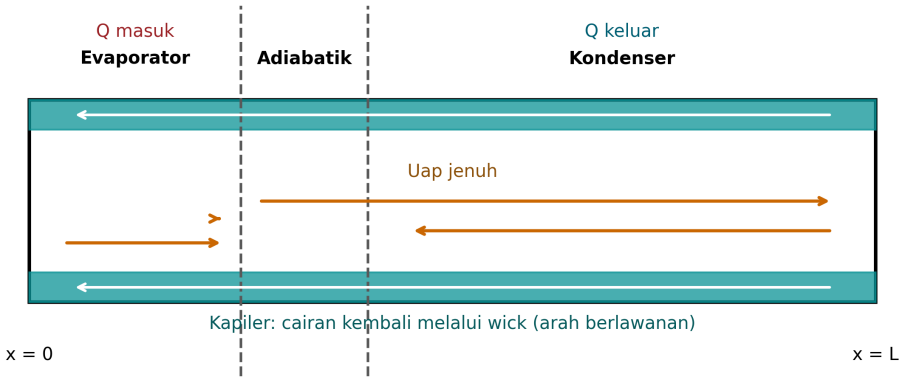
Penelitian ini berada pada tingkat model analitik–numerik tereduksi. Kebaruannya tidak diklaim sebagai penemuan persamaan baru, melainkan pada integrasi paket tugas akhir yang menghubungkan empat komponen: korelasi sifat natrium yang dapat diuji, profil aksial uap–cair, pencarian batas kapiler melalui biseksi, serta keluaran Quarto yang dapat direproduksi. Parameter percepatan efektif digunakan untuk membandingkan mikrogravitasi dengan orientasi dan percepatan lain.

Kebaruan pada tingkat tugas akhir juga terletak pada keterbukaan implementasi. Setiap rumus utama memiliki padanan fungsi, keluaran disimpan dalam format CSV dan JSON, dan pengujian otomatis disertakan. Dengan demikian, pembaca dapat mengubah parameter serta menilai konsekuensinya tanpa bergantung pada tabel statis. Model ini tetap merupakan penyederhanaan dan tidak dimaksudkan menggantikan kode tervalidasi untuk desain reaktor.

BAB III
DASAR TEORI

III.1. Deskripsi Model dan Volume Kontrol

Pipa kalor dipandang sebagai volume kontrol satu dimensi dengan koordinat x dari ujung tertutup evaporator menuju ujung kondenser. Panjang total L merupakan jumlah panjang evaporator L_e, bagian adiabatik L_a, dan kondenser L_c. Ruang uap memiliki luas A_v, sedangkan luas annulus sumbu cair adalah A_w. Gambar III.1 memperlihatkan arah aliran yang berlawanan: uap bergerak dari evaporator ke kondenser dan cairan kembali melalui sumbu.



Gambar III.1: Skema volume kontrol dan jalur aliran uap–cair yang digunakan dalam model. Garis putus-putus menunjukkan batas evaporator, adiabatik, dan kondenser.

Asumsi kesetimbangan jenuh memungkinkan sifat cairan dan uap dievaluasi pada temperatur operasi T_v. Model tidak menyelesaikan distribusi temperatur dinding atau antarmuka, sehingga T_v adalah temperatur efektif untuk perhitungan hidraulik. Resistansi kondenser menghubungkan temperatur tersebut dengan temperatur sink.

III.2. Neraca Energi dan Massa

Dengan kehilangan kalor pada bagian adiabatik diabaikan, daya yang masuk evaporator sama dengan daya yang dilepas kondenser. Temperatur operasi didekati dengan

$$T_v = T_{sink} + R_c Q, \tag{3.1}$$

dengan Q adalah daya masukan, R_c resistansi termal efektif kondenser, dan T_{sink} temperatur sink. Pendekatan ini adalah kondisi batas sederhana, bukan model radiator. Nilai R_c harus diganti apabila radiasi atau pertukaran kalor diselesaikan

secara lebih rinci.

Laju aliran massa lokal berkaitan dengan kalor aksial $Q_x(x)$ melalui kalor laten:

$$\dot{m}(x) = \frac{Q_x(x)}{h_{fg}}. \tag{3.2}$$

Untuk pemanasan dan pendinginan merata, profil kalor aksial adalah

$$Q_x(x) = \begin{cases} Qx/L_e, & 0 \le x \le L_e, \\ Q, & L_e < x \le L_e + L_a, \\ Q\left[1 - (x - L_e - L_a)/L_c\right], & L_e + L_a < x \le L. \end{cases} \tag{3.3}$$

Profil tersebut bernilai nol pada kedua ujung dan maksimum sebesar Q di antara evaporator dan kondenser. Integral pembangkitan uap di evaporator sama dengan integral kondensasi di kondenser sehingga neraca massa global terpenuhi secara konstruksi.

III.3. Sifat Natrium

Tekanan jenuh natrium dihitung dengan korelasi Browning–Potter yang direkomendasikan Fink dan Leibowitz [7]:

$$\ln P = 11{,}9463 - \frac{12633{,}73}{T_v} - 0{,}4672 \ln T_v, \tag{3.4}$$

dengan P dalam MPa dan T_v dalam K. Program mengonversi tekanan ke Pa. Entalpi penguapan dihitung dengan

$$h_{fg} = 393{,}37(1 - T_v/T_{cr}) + 4398{,}6(1 - T_v/T_{cr})^{0{,}29302}, \tag{3.5}$$

dengan h_{fg} dalam kJ/kg dan $T_{cr} = 2503{,}7$ K. Densitas cairan dalam kg/m³ adalah

$$\rho_l = 219 + 275{,}32\tau + 511{,}58\tau^{1/2}, \qquad \tau = 1 - T_v/T_{cr}. \tag{3.6}$$

Viskositas cairan dalam Pa·s dihitung dari

$$\ln \mu_l = -6{,}4406 - 0{,}3958 \ln T_v + \frac{556{,}835}{T_v}. \tag{3.7}$$

Tegangan permukaan mengikuti bentuk Van der Waals:

$$\sigma = 0{,}2405(1 - T_v/T_{cr})^{1{,}126}. \tag{3.8}$$

Densitas uap didekati dengan gas ideal,

$$\rho_v = \frac{PM}{R_u T_v}, \tag{3.9}$$

dengan massa molar natrium $M = 0{,}02298977$ kg/mol. Viskositas uap menggunakan estimasi gas encer bola keras karena laporan sifat yang sama tidak memberikan korelasi viskositas uap. Pendekatan ini merupakan sumber ketidakpastian dan perlu diganti untuk analisis berketelitian tinggi.

III.4. Aliran Uap

Kecepatan rata-rata uap pada setiap posisi adalah

$$u_v(x) = \frac{\dot{m}(x)}{\rho_v A_v}. \tag{3.10}$$

Bilangan Reynolds dan Mach dinyatakan sebagai

$$Re_v = \frac{\rho_v u_v D_v}{\mu_v}, \qquad Ma_v = \frac{u_v}{a_v}, \tag{3.11}$$

dengan $a_v = \sqrt{\gamma R_v T_v}$ dan $\gamma = 5/3$ untuk pendekatan gas monoatomik. Faktor gesek Darcy ditentukan dengan $64/Re_v$ untuk $Re_v < 2300$ dan korelasi Blasius $0{,}3164 Re_v^{-0{,}25}$ untuk kondisi di luar rentang tersebut. Gradien rugi tekanan uap adalah

$$\frac{d(\Delta P_v)}{dx} = \frac{f}{D_v}\frac{\rho_v u_v^2}{2}. \tag{3.12}$$

Persamaan (3.12) diintegrasikan sepanjang pipa. Bilangan Mach digunakan sebagai pemeriksaan penerimaan pendekatan tak kompresibel; kasus acuan mensyaratkan $Ma < 0{,}3$. Apabila batas tersebut terlampaui, hasil harus diinterpretasikan sebagai penyaringan dan model kompresibel diperlukan.

III.5. Aliran Balik Cairan dan Tekanan Kapiler

Aliran cairan di dalam medium berpori mengikuti hukum Darcy. Untuk arah berlawanan dengan uap, besar gradien rugi tekanannya adalah

$$\frac{d(\Delta P_l)}{dx} = \frac{\mu_l \dot{m}(x)}{\rho_l K A_w}. \tag{3.13}$$

Untuk profil kalor pada Persamaan (3.3), integrasi menghasilkan panjang efektif

$$L_{eff} = \frac{L_e}{2} + L_a + \frac{L_c}{2}. \tag{3.14}$$

Dengan demikian, rugi tekanan cairan total dapat pula ditulis dalam bentuk kompak

$$\Delta P_l = \frac{\mu_l Q L_{eff}}{\rho_l K A_w h_{fg}}. \tag{3.15}$$

Tekanan kapiler maksimum didekati dengan persamaan Young–Laplace:

$$\Delta P_{cap} = \frac{2\sigma \cos\theta_c}{r_c}. \tag{3.16}$$

Kondisi operasi berkelanjutan adalah

$$\Delta P_{cap} \geq \Delta P_v + \Delta P_l + \Delta P_g, \tag{3.17}$$

dengan tekanan akibat percepatan efektif

$$\Delta P_g = \rho_l g_{eff} L \sin\theta. \tag{3.18}$$

Sudut θ adalah kemiringan terhadap arah yang menghasilkan komponen hidrostatik aksial. Pada kasus mikrogravitasi ideal, $g_{eff} = 0$ sehingga $\Delta P_g = 0$. Model juga menerima percepatan nonnol untuk meniru residu akselerasi atau membandingkan kondisi Bumi. Nilai nol pada mikrogravitasi ideal tidak berarti seluruh masalah dua fase menghilang; kapilaritas, gesekan, dan inersia uap tetap diperhitungkan.

III.6. Batas Transportasi Kalor

Batas kapiler Q_{cap} adalah daya yang membuat ruas kanan Persamaan (3.17) sama dengan ΔP_{cap}. Karena rugi tekanan uap tidak selalu linear terhadap Q, program mencari akar margin kapiler dengan biseksi. Apabila tekanan hidrostatik saja

sudah melebihi tekanan kapiler, batas kapiler ditetapkan nol sebagai indikator ketidakmampuan model beroperasi pada kondisi tersebut.

Batas sonik disaring menggunakan bentuk yang lazim dalam literatur pipa kalor [3, 4]:

$$Q_s = 0{,}474 A_v h_{fg} \sqrt{\rho_v P}. \tag{3.19}$$

Batas entrainment didekati dengan

$$Q_{ent} = A_v h_{fg} \sqrt{\frac{\rho_v \sigma}{2 r_c}}. \tag{3.20}$$

Batas pendidihan di dalam sumbu dihitung sebagai

$$Q_b = \frac{2\pi L_e k_{eff} T_v}{h_{fg} \rho_v \ln(r_i / r_v)} \left(\frac{2\sigma}{r_b} - \Delta P_v - \Delta P_l - \Delta P_g \right), \tag{3.21}$$

dengan k_{eff} konduktivitas efektif sumbu, r_b jari-jari nukleasi, r_v jari-jari ruang uap, dan $r_i = r_v + t_w$. Ketiga persamaan terakhir bersifat penyaringan dengan ketidakpastian korelasi. Nilai terkecil dari keempat batas digunakan sebagai batas pengatur untuk perbandingan, tetapi bukan jaminan bahwa seluruh mekanisme kegagalan telah tercakup.

III.7. Metode Numerik

Domain dibagi menjadi N sel dengan titik $x_i = iL/N$. Besaran lokal dihitung pada setiap titik. Integrasi rugi tekanan menggunakan aturan trapesium:

$$\int_0^L y(x)\,dx \approx \sum_{i=0}^{N-1} \frac{y_i + y_{i+1}}{2} (x_{i+1} - x_i). \tag{3.22}$$

Untuk mencari Q_{cap}, algoritme pertama memeriksa margin pada $Q = 0$. Jika margin tidak positif, kapasitas kapiler adalah nol. Jika positif, batas atas daya digandakan sampai margin berubah tanda. Biseksi kemudian mempersempit interval. Skema ini dipilih karena margin menurun secara monoton pada rentang yang dianalisis dan tidak membutuhkan turunan.

III.8. Hipotesis

Berdasarkan persamaan di atas, dirumuskan dua dugaan yang diuji secara numerik. Pertama, pada geometri acuan, rugi tekanan cairan lebih besar daripada rugi tekanan uap sehingga batas kapiler menjadi pengatur pada temperatur sekitar 1000 K. Kedua, peningkatan percepatan efektif pada orientasi merugikan menurunkan

kapasitas kapiler, sedangkan mikrogravitasi ideal menghilangkan kontribusi hidrostatik. Dugaan ini hanya berlaku untuk parameter yang diuji dan tidak digeneralisasi ke seluruh desain pipa kalor.

BAB IV
PELAKSANAAN PENELITIAN

IV.1. Alat dan Bahan Penelitian

Penelitian menggunakan perangkat lunak dan data sifat fluida; tidak dilakukan eksperimen laboratorium. Simulator ditulis dalam Python 3.12. NumPy 2.2.5 digunakan untuk larik dan operasi numerik, sedangkan Matplotlib 3.10.3 digunakan untuk grafik [14, 15]. Pustaka standar `dataclasses`, `csv`, `json`, `argparse`, dan `unittest` digunakan untuk struktur parameter, keluaran, antarmuka perintah, dan pengujian. Dokumen disusun dalam format Quarto Markdown dengan keluaran PDF berbasis LaTeX.

Struktur proyek dipisahkan menjadi sumber pada `src/`, pengujian pada `tests/`, bab pada `chapters/`, lampiran pada `appendices/`, data pada `data/`, dan gambar pada `figures/`. Pemisahan tersebut mencegah pencampuran antara persamaan, keluaran numerik, dan naskah. Nama berkas keluaran menggunakan bahasa Indonesia agar mudah dikenali dalam pembahasan.

Parameter geometri dan kondisi acuan diberikan pada Tabel IV.1. Geometri bukan salinan pipa kalor reaktor tertentu, melainkan kasus representatif yang dipilih agar model memperlihatkan mekanisme kapiler, uap, dan gravitasi dalam satu rangkaian perhitungan.

Tabel IV.1: Parameter geometri, sumbu, dan kondisi batas kasus acuan.

Parameter	Simbol	Nilai	Satuan
Panjang evaporator	L_e	0,25	m
Panjang adiabatik	L_a	0,15	m
Panjang kondenser	L_c	0,60	m
Diameter ruang uap	D_v	0,012	m
Tebal sumbu	t_w	0,001	m
Jari-jari pori efektif	r_c	75	µm
Permeabilitas	K	5×10^{-10}	m²
Sudut kontak	θ_c	0	°
Konduktivitas efektif sumbu	k_{eff}	10	W/(m·K)
Jari-jari nukleasi	r_b	5	µm
Daya masukan	Q	800	W
Temperatur sink	T_{sink}	700	K
Resistansi kondenser	R_c	0,375	K/W

Parameter	Simbol	Nilai	Satuan
Percepatan efektif	g_{eff}	0	m/s²
Jumlah sel	N	400	–

Nilai turunan yang digunakan adalah panjang total 1,00 m, panjang efektif 0,575 m, luas ruang uap $1{,}13097 \times 10^{-4}$ m², dan luas sumbu $4{,}08407 \times 10^{-5}$ m². Sifat natrium dihitung pada setiap temperatur operasi, bukan diambil sebagai konstanta.

IV.2. Tata Laksana Penelitian

Tahapan penelitian dilaksanakan sebagai berikut.

1. Menurunkan neraca energi, profil kalor aksial, laju aliran massa, rugi tekanan uap, rugi tekanan cairan, dan neraca kapiler.
2. Mengimplementasikan korelasi sifat natrium pada kelas `SodiumProperties` dengan pemeriksaan rentang temperatur.
3. Menyusun parameter pada kelas data `HeatPipeGeometry` dan `OperatingConditions` agar masukan dapat divalidasi dan diubah tanpa mengedit persamaan.
4. Membentuk profil kalor aksial sesuai Persamaan (3.3) dan menghitung kecepatan, Reynolds, serta Mach.
5. Mengintegrasikan rugi tekanan dengan aturan trapesium dan mengevaluasi margin kapiler.
6. Menghitung batas sonik, entrainment, dan pendidihan pada temperatur operasi.
7. Mencari batas kapiler melalui bracketing dan biseksi pada temperatur tetap.
8. Menjalankan kasus acuan, peta batas pada 800–1200 K, studi beban, studi percepatan, studi pori, dan konvergensi grid.
9. Menyimpan data dalam CSV/JSON dan membuat gambar langsung dari hasil perhitungan.
10. Menjalankan pengujian otomatis dan memeriksa konsistensi antara angka dalam naskah dan berkas keluaran.

Perintah reproduksi dari direktori utama proyek adalah sebagai berikut.

```
python -m pip install -r requirements.txt
python src/heatpipe_simulator.py
python -m unittest discover -s tests -v
quarto render Skripsi.qmd --to pdf
```

Program tidak membutuhkan koneksi internet pada saat simulasi. Instalasi

paket dan renderer hanya diperlukan pada penyiapan awal. Seluruh keluaran menggunakan satuan SI secara internal; pemformatan desimal pada naskah dilakukan ketika hasil disajikan.

IV.3. Rencana Analisis Hasil Penelitian

Hasil utama dirangkum dalam enam kelompok. Pertama, sifat fluida pada 1000 K diperiksa terhadap nilai acuan. Kedua, kasus acuan dianalisis melalui laju massa maksimum, kecepatan, Reynolds, Mach, rugi tekanan, margin kapiler, dan empat batas operasi. Ketiga, peta batas terhadap temperatur digunakan untuk mengidentifikasi mekanisme pengatur.

Keempat, studi beban dilakukan dengan mempertahankan T_{sink} dan R_c, sehingga temperatur operasi berubah mengikuti Persamaan (3.1). Analisis ini tidak sama dengan peta pada temperatur tetap. Kelima, studi percepatan dilakukan pada 1000 K untuk $g_{eff} = 0$, 0,01, 0,1, 1, dan 9,81 m/s². Orientasi 90° digunakan untuk kasus nonnol agar seluruh komponen aksial bekerja merugikan. Keenam, jari-jari pori divariasikan dengan permeabilitas tetap untuk mengisolasi pengaruh tekanan kapiler dan entrainment.

Konvergensi grid dinilai dari jumlah rugi tekanan uap dan cairan pada $N = 50$ sampai 1600. Grid 1600 digunakan sebagai pembanding relatif. Karena profil kalor aksial merupakan fungsi linear per bagian, integrasi trapesium diperkirakan menjadi sangat tepat ketika batas bagian berimpit dengan titik grid.

Verifikasi program dilakukan melalui sepuluh pengujian yang tercantum pada Tabel IV.2. Pengujian ini memeriksa implementasi dan konsistensi fisik, tetapi tidak menggantikan validasi eksperimental.

Tabel IV.2: Rancangan verifikasi perangkat lunak.

Kelompok	Hal yang diuji	Kriteria
Sifat fluida	Tekanan jenuh pada titik didih normal	Sekitar 1 atm pada 1156 K
Sifat fluida	σ, μ_l, h_{fg}, dan ρ_l pada 1000 K	Sesuai nilai acuan/toleransi
Sifat fluida	Semua sifat pada 1000 K	Positif dan berhingga
Profil energi	Nilai ujung, maksimum, dan arah gradien	Nol–maksimum–nol sesuai geometri
Mikrogravitasi	Tekanan hidrostatik	Nol ketika $g_{eff} = 0$
Kasus acuan	Mekanisme pengatur dan Mach	Kapiler dan $Ma < 0,3$

Kelompok	Hal yang diuji	Kriteria
Batas kapiler	Margin pada $Q = Q_{cap}$	Mendekati nol
Percepatan	Kapasitas pada orientasi merugikan	Menurun; kasus Bumi acuan tidak feasible
Grid	Rugi tekanan grid 100 dan 1600	Selisih relatif <0,5%
Validasi masukan	Permeabilitas nol	Ditolak oleh program

BAB V
HASIL DAN PEMBAHASAN

V.1. Hasil Kasus Acuan

Kasus acuan menghasilkan temperatur operasi 1000 K dari hubungan $T_v = 700 + 0{,}375(800)$. Sifat fluida yang digunakan pada temperatur tersebut disajikan pada Tabel V.1. Tekanan jenuh sebesar 19,95 kPa dan kalor laten sekitar 4,02 MJ/kg menghasilkan laju aliran massa maksimum $1{,}98784 \times 10^{-4}$ kg/s.

Tabel V.1: Sifat natrium pada temperatur operasi 1000 K.

Besaran	Nilai	Satuan
Tekanan jenuh	19.945,87	Pa
Kalor laten	4.024.461,18	J/kg
Densitas cairan	780,818	kg/m³
Densitas uap	0,055151	kg/m³
Viskositas cairan	$1{,}80848 \times 10^{-4}$	Pa·s
Viskositas uap	$2{,}52981 \times 10^{-5}$	Pa·s
Tegangan permukaan	0,135455	N/m
Konduktivitas cairan	54,244	W/(m·K)
Kecepatan suara pendekatan	776,380	m/s

Hasil hidraulik utama diberikan pada Tabel V.2. Rugi tekanan cairan sebesar 1296,43 Pa jauh lebih besar daripada rugi tekanan uap sebesar 103,02 Pa. Perbandingan ini konsisten dengan dominasi hambatan viskos pada kanal berpori. Tekanan kapiler yang tersedia adalah 3612,13 Pa sehingga tersisa margin 2212,68 Pa. Tidak ada tekanan hidrostatik karena percepatan efektif ditetapkan nol.

Tabel V.2: Ringkasan hasil kasus acuan 800 W pada mikrogravitasi.

Besaran	Nilai
Laju massa maksimum	$1{,}98784 \times 10^{-4}$ kg/s
Kecepatan uap maksimum	31,8696 m/s
Bilangan Reynolds maksimum	833,724
Bilangan Mach maksimum	0,04105
Tekanan kapiler	3612,13 Pa
Rugi tekanan uap	103,02 Pa

Besaran	Nilai
Rugi tekanan cairan	1296,43 Pa
Rugi tekanan gravitasi	0 Pa
Margin kapiler	2212,68 Pa
Utilisasi batas pengatur	38,74%

Bilangan Reynolds maksimum berada dalam rentang laminar pada model dan bilangan Mach jauh di bawah 0,3. Oleh karena itu, penggunaan pendekatan rugi tekanan tak kompresibel dapat diterima untuk kasus ini sebagai penyaringan. Pernyataan tersebut tidak berlaku otomatis pada temperatur rendah ketika densitas uap jauh lebih kecil.

V.2. Profil Aksial

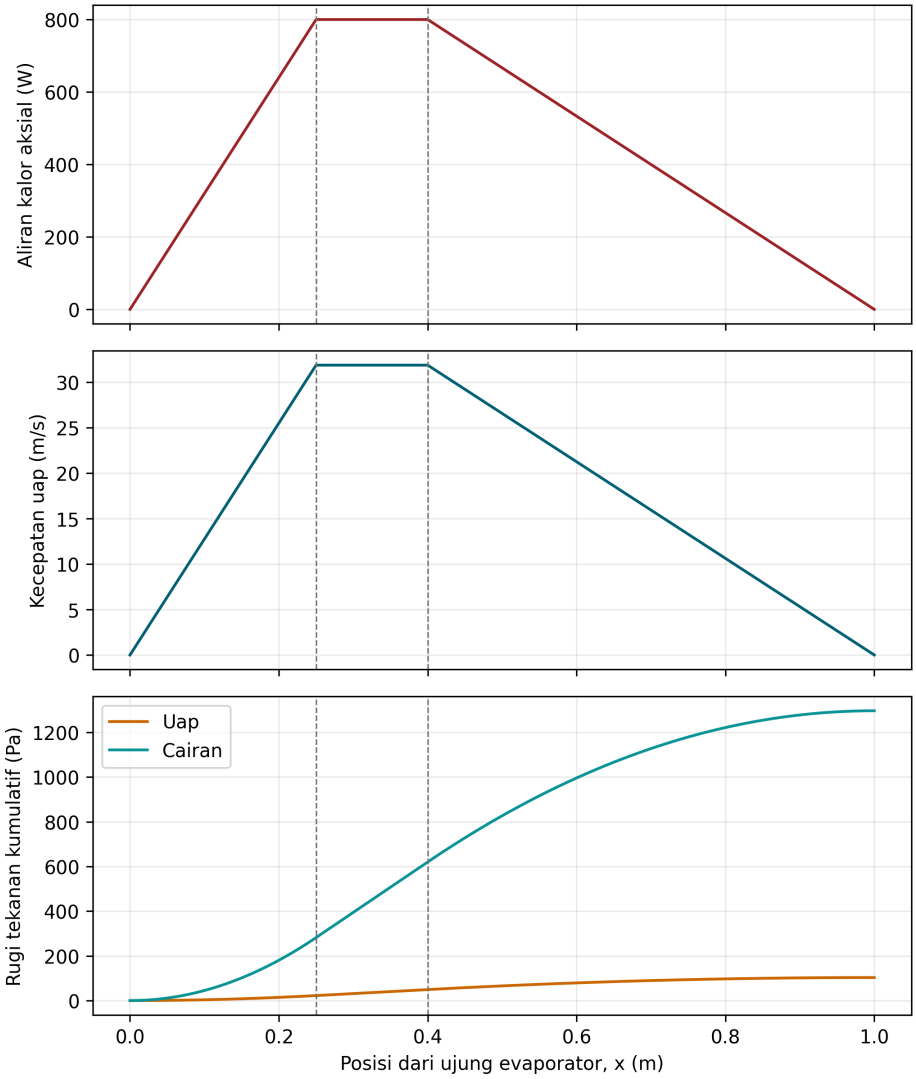
Gambar V.1 menunjukkan kalor aksial meningkat linear di evaporator, konstan pada bagian adiabatik, lalu menurun linear di kondenser. Kecepatan uap mengikuti bentuk yang sama karena sifat fluida dan luas penampang konstan pada satu perhitungan. Nilai maksimum terjadi di sepanjang bagian adiabatik, yaitu antara $x = 0,25$ m dan $x = 0,40$ m.

Rugi tekanan kumulatif bertambah paling cepat di bagian adiabatik karena laju massa berada pada nilai maksimum. Di evaporator dan kondenser, gradien berubah mengikuti profil laju massa. Rugi tekanan cairan memiliki bentuk serupa dengan skala yang lebih besar. Hasil ini menegaskan bahwa panjang efektif dalam Persamaan (3.14) bukan sekadar panjang total; distribusi pemanasan dan pendinginan ikut menentukan integrasi aliran.

V.3. Batas Operasi dan Pengaruh Temperatur

Pada kasus acuan, batas kapiler adalah 2064,88 W, batas sonik 7155,53 W, batas entrainment 3212,10 W, dan batas pendidihan 24.232,80 W. Batas kapiler merupakan nilai terkecil sehingga menjadi pengatur. Daya 800 W menggunakan 38,74% dari batas tersebut. Batas pendidihan bergantung pada rugi tekanan kasus aktual, sedangkan peta temperatur menggunakan proba daya kecil agar perbandingan sifat lebih konsisten; karena itu terdapat perbedaan kecil antara nilai pendidihan pada peta dan kasus acuan.

Tabel V.3 dan Gambar V.2 memperlihatkan perubahan mekanisme pengatur. Pada 800 K batas sonik hanya 393,48 W, lebih rendah daripada batas kapiler dan entrainment. Pada 900 K, batas entrainment sedikit lebih rendah daripada batas

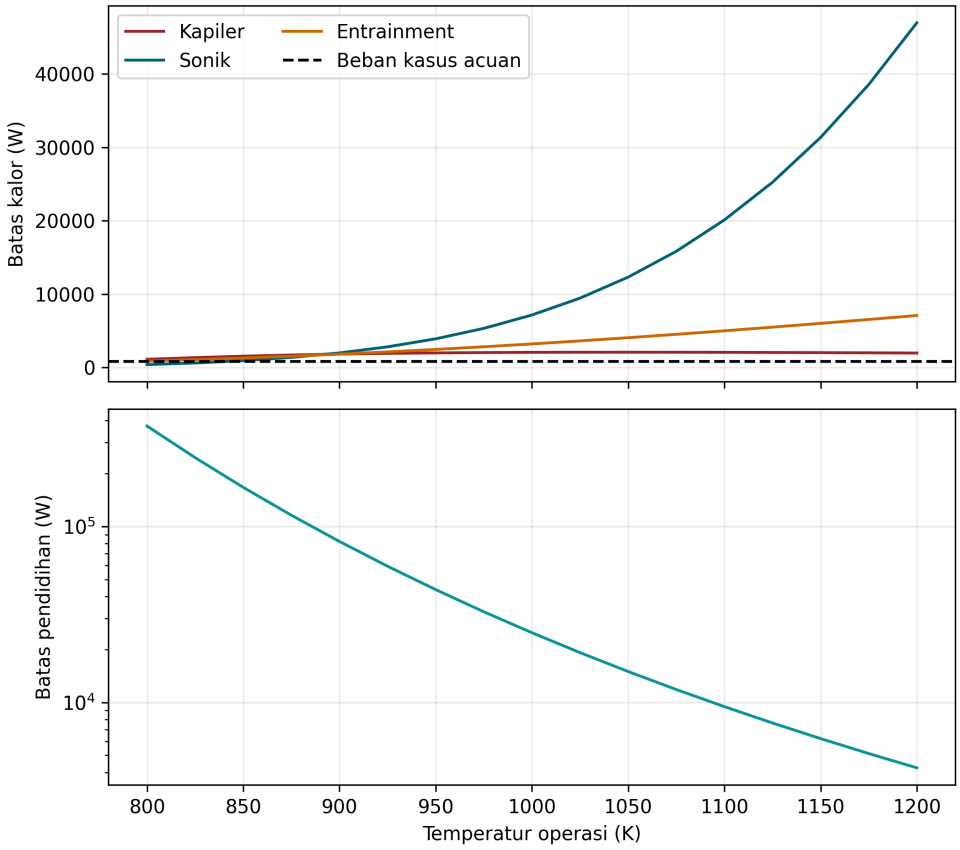


Gambar V.1: Profil kalor aksial, kecepatan uap, dan rugi tekanan kumulatif untuk kasus acuan. Posisi $x = 0$ adalah ujung evaporator; garis batas berada pada 0,25 m dan 0,40 m.

kapiler. Pada 1000–1200 K, batas kapiler menjadi pengatur. Peningkatan temperatur menaikkan tekanan dan densitas uap sehingga batas sonik meningkat tajam. Sebaliknya, tegangan permukaan menurun terhadap temperatur dan pada akhirnya mengurangi tekanan kapiler.

Tabel V.3: Peta batas transportasi kalor pada temperatur tetap.

Temperatur (K)	Kapiler (W)	Sonik (W)	Entrainment (W)	Pendidihan (W)	Pengatur
800	1119,89	393,48	872,58	372.531,62	Sonik
900	1828,89	1989,01	1822,46	82.149,60	Entrainment
1000	2064,88	7155,53	3212,10	24.874,49	Kapiler
1100	2066,85	20.110,97	5000,63	9449,29	Kapiler
1200	1969,78	46.978,71	7086,28	4241,04	Kapiler



Gambar V.2: Peta batas operasi terhadap temperatur. Panel atas menampilkan batas kapiler, sonik, entrainment, dan beban acuan; panel bawah menampilkan batas pendidihan dalam skala logaritmik.

Batas kapiler meningkat dari 800 K hingga sekitar 1050–1100 K kemudian menurun. Perilaku nonmonoton tersebut muncul dari kompetisi beberapa sifat: kenaikan kalor laten atau densitas uap memengaruhi rugi tekanan dan laju massa, sedangkan viskositas cairan serta tegangan permukaan menurun. Model tidak men-

dukung kesimpulan bahwa temperatur yang lebih tinggi selalu meningkatkan kapasitas. Selain itu, hasil di bawah rentang rekomendasi sifat atau di dekat kondisi sonik harus diperlakukan dengan kehati-hatian.

V.4. Pengaruh Beban Kalor

Studi beban menggunakan resistansi kondenser tetap sehingga setiap kenaikan daya juga menaikkan temperatur operasi. Hasil pada Tabel V.4 memperlihatkan dua pengaruh yang berlawanan. Kenaikan daya meningkatkan laju massa yang diperlukan, tetapi kenaikan temperatur meningkatkan densitas uap dan dapat menurunkan kecepatan serta Mach. Akibatnya, Mach maksimum justru menurun dari 0,223 pada 300 W menjadi 0,00617 pada 1600 W. Perilaku ini merupakan konsekuensi kondisi batas $T_v = T_{sink} + R_c Q$, bukan karakter universal pipa kalor.

Tabel V.4: Studi beban dengan temperatur sink dan resistansi kondenser tetap.

Daya (W)	Temperatur (K)	Pengatur	Utilisasi (%)	Mach maksimum	Margin kapiler (Pa)
300	812,5	Sonik	60,85	0,22343	3121,80
600	925,0	Kapiler	31,20	0,07824	2625,15
800	1000,0	Kapiler	38,74	0,04105	2212,68
1200	1150,0	Kapiler	59,21	0,01405	1308,82
1600	1300,0	Kapiler	87,75	0,00617	344,51

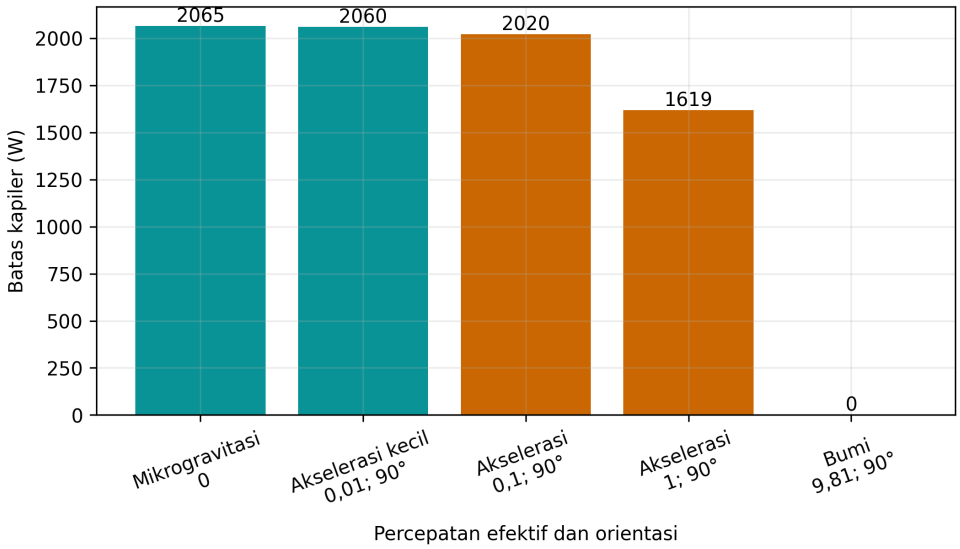
Margin kapiler menurun ketika beban menjadi besar dan utilisasi mencapai 87,75% pada 1600 W. Oleh sebab itu, peningkatan beban tetap mendekatkan sistem pada keterbatasan meskipun Mach menurun. Hasil ini memperlihatkan bahwa pemeriksaan satu indikator tidak cukup untuk menilai kelayakan operasi.

V.5. Pengaruh Percepatan Efektif

Gambar V.3 dan Tabel V.5 menunjukkan batas kapiler pada 1000 K untuk beberapa percepatan. Percepatan 0,01 m/s² pada orientasi 90° hanya menurunkan kapasitas dari 2064,88 W menjadi 2060,42 W. Penurunan menjadi lebih besar pada 1 m/s², yaitu 1618,53 W. Pada percepatan 9,81 m/s² dengan orientasi merugikan, tekanan hidrostatik melebihi tekanan kapiler sehingga tidak ada kapasitas positif menurut model.

Tabel V.5: Pengaruh percepatan efektif pada batas kapiler pada 1000 K.

Percepatan (m/s²)	Orientasi	Batas kapiler (W)
0	0°	2064,88
0,01	90°	2060,42
0,1	90°	2020,25
1	90°	1618,53
9,81	90°	0



Gambar V.3: Perbandingan batas kapiler pada mikrogravitasi, beberapa akselerasi, dan kondisi Bumi dengan orientasi merugikan.

Hasil kasus Bumi tidak berarti pipa kalor natrium secara umum tidak dapat bekerja pada orientasi vertikal. Kesimpulan hanya berlaku untuk geometri, sumbu, dan orientasi acuan. Pipa dengan pori lebih kecil, permeabilitas lebih tinggi, panjang lebih pendek, atau orientasi menguntungkan dapat memiliki perilaku berbeda. Di sisi lain, hasil mikrogravitasi memperlihatkan alasan kapilaritas penting: sirkulasi tetap dimungkinkan tanpa kontribusi gravitasi selama neraca Persamaan (3.17) terpenuhi.

V.6. Pengaruh Jari-Jari Pori

Tabel V.6 menunjukkan bahwa pembesaran pori dari 25 µm menjadi 150 µm menurunkan batas kapiler dari 5544,26 W menjadi 1032,44 W ketika permeabilitas dibuat tetap. Batas entrainment juga menurun, tetapi dengan laju berbeda. Pada pori 25 µm, batas entrainment sedikit lebih tinggi daripada batas kapiler sehingga

kapiler masih sedikit lebih kecil; perbedaan keduanya sangat dekat dan tidak boleh ditafsirkan melebihi ketidakpastian korelasi.

Tabel V.6: Pengaruh jari-jari pori dengan permeabilitas dipertahankan tetap.

Jari-jari pori (µm)	Batas kapiler (W)	Batas entrainment (W)
25	5544,26	5563,52
50	2974,82	3934,00
75	2064,88	3212,10
100	1548,66	2781,76
150	1032,44	2271,30

Hasil ini tidak dapat digunakan untuk menyimpulkan bahwa pori sekecil mungkin selalu optimal. Pada material nyata, pengecilan pori dapat menurunkan permeabilitas dan meningkatkan rugi tekanan cairan. Model sengaja mempertahankan K untuk isolasi parameter; studi desain selanjutnya harus menggunakan korelasi eksperimen yang menghubungkan K, r_c, porositas, dan struktur sumbu.

V.7. Konvergensi Grid dan Verifikasi

Seluruh sepuluh pengujian otomatis berhasil. Pengujian tekanan jenuh pada 1156 K berada dalam 3% dari 1 atm, sifat pada 1000 K sesuai toleransi terhadap nilai acuan, dan margin kapiler pada daya batas mendekati nol. Program juga menolak permeabilitas nonpositif. Hasil konvergensi diberikan pada Tabel V.7.

Tabel V.7: Konvergensi jumlah rugi tekanan uap dan cairan pada kasus acuan.

Jumlah sel	Rugi tekanan total (Pa)	Selisih relatif terhadap 1600 sel (%)
50	1398,9667	0,03478
100	1399,4535	$< 10^{-10}$
200	1399,4535	$< 10^{-10}$
400	1399,4535	$< 10^{-10}$
800	1399,4535	$< 10^{-10}$
1600	1399,4535	0

Perbedaan terbesar hanya 0,03478% pada 50 sel dan hampir menghilang setelah 100 sel. Hasil ini masuk akal karena integrand berasal dari profil linear per bagian dan grid 100 sel atau lebih menempatkan batas geometri tepat pada titik integrasi. Konvergensi tersebut menunjukkan konsistensi diskretisasi, bukan bukti bahwa model telah sesuai dengan fisika eksperimen.

V.8. Pembahasan Keterbatasan Model

Model memberikan hasil yang konsisten dengan dua hipotesis: rugi tekanan cairan dominan pada kasus acuan dan percepatan merugikan menurunkan kapasitas kapiler. Namun, beberapa keterbatasan menentukan interpretasi hasil. Pertama, temperatur uap seragam menghilangkan gradien temperatur aksial dan resistansi evaporator. Kedua, gas ideal dan viskositas uap bola keras merupakan penyederhanaan. Ketiga, persamaan faktor gesek tidak menyelesaikan aliran kompresibel, pusaran, atau interaksi gesek antarmuka.

Keempat, kondisi jenuh sumbu mengabaikan inventor cairan dan kemungkinan pengeringan lokal sebelum batas global. Kelima, model pendidihan sangat sensitif terhadap jari-jari nukleasi dan konduktivitas efektif yang tidak dikalibrasi. Keenam, start-up beku dan gas takterkondensasi tidak dicakup meskipun keduanya penting untuk pipa kalor alkali. Ketujuh, kasus tidak dikopelkan dengan teras reaktor, radiator, atau umpan balik reaktivitas.

Dengan demikian, istilah yang tepat untuk tahap ini adalah **verifikasi kode dan studi numerik**, bukan validasi desain. Validasi memerlukan data temperatur, daya, tekanan, geometri sumbu, dan kondisi orientasi dari eksperimen yang terdokumentasi. Ketidakpastian parameter juga perlu dianalisis sebelum hasil digunakan untuk menetapkan margin keselamatan.

BAB VI
KESIMPULAN DAN SARAN

VI.1. Kesimpulan

Berdasarkan pengembangan model dan hasil perhitungan, diperoleh kesimpulan sebagai berikut.

1. Simulator aliran dua fase tunak satu dimensi telah berhasil disusun dalam Python dengan menggabungkan neraca energi, profil laju massa, aliran uap, aliran balik cairan Darcy, dan tekanan kapiler. Struktur kelas data, keluaran CSV/JSON, gambar otomatis, serta dokumen Quarto memungkinkan perhitungan direproduksi.
2. Pada kasus acuan 800 W dan 1000 K dalam mikrogravitasi, rugi tekanan uap dan cairan masing-masing adalah 103,02 Pa dan 1296,43 Pa. Tekanan kapiler 3612,13 Pa memberikan margin 2212,68 Pa. Kecepatan uap maksimum adalah 31,87 m/s dan Mach maksimum 0,04105.
3. Batas kapiler sebesar 2064,88 W menjadi batas pengatur kasus acuan. Batas sonik, entrainment, dan pendidihan masing-masing adalah 7155,53 W, 3212,10 W, dan 24.232,80 W. Utilisasi daya terhadap batas pengatur adalah 38,74%.
4. Peta temperatur menunjukkan batas sonik mengatur pada 800 K, entrainment sedikit mengatur pada 900 K, dan kapiler mengatur pada 1000–1200 K. Kapasitas kapiler mencapai sekitar 2066 W di dekat 1100 K kemudian menurun menjadi 1969,78 W pada 1200 K.
5. Pada orientasi merugikan dan temperatur 1000 K, percepatan 1 m/s^2 menurunkan batas kapiler menjadi 1618,53 W. Kondisi Bumi 9,81 m/s^2 menghasilkan tekanan hidrostatik yang melampaui kapiler untuk geometri acuan, sedangkan mikrogravitasi ideal menghilangkan rugi tekanan tersebut.
6. Sepuluh pengujian otomatis berhasil dan perbedaan rugi tekanan antara grid 50 dan 1600 sel hanya 0,03478%. Hasil ini memverifikasi konsistensi implementasi, tetapi belum merupakan validasi eksperimental.

VI.2. Saran

Pengembangan selanjutnya perlu diarahkan pada validasi terhadap data eksperimen pipa kalor natrium dengan geometri dan kondisi batas yang terdokumentasi. Model sifat uap sebaiknya ditingkatkan dengan korelasi yang sesuai untuk campuran monomer dan dimer, sedangkan aliran kompresibel perlu digunakan pada kasus bermach tinggi. Hubungan eksperimen antara pori, permeabilitas, porositas, dan

konduktivitas efektif juga diperlukan agar studi desain lebih realistis.

Model dapat dikembangkan menjadi jaringan termal transien yang mencakup kapasitas panas dinding, start-up beku, gas takterkondensasi, dan radiator. Untuk analisis reaktor, kopling dengan model teras dan umpan balik reaktivitas harus dilakukan setelah model pipa tervalidasi. Analisis ketidakpastian dan sensitivitas global disarankan sebelum hasil digunakan untuk menetapkan margin desain. Terakhir, studi kegagalan satu pipa dan propagasinya pada tingkat sistem merupakan kelanjutan penting yang tidak dapat dijawab oleh model satu pipa ini.

DAFTAR PUSTAKA

[1] G. M. Grover, T. P. Cotter, dan G. F. Erickson. "Structures of Very High Thermal Conductance." *Journal of Applied Physics*, vol. 35, no. 6, pp. 1990–1991, 1964. DOI: 10.1063/1.1713792.

[2] T. P. Cotter. *Theory of Heat Pipes*. LA-3246-MS, Los Alamos Scientific Laboratory, 1965.

[3] S. W. Chi. *Heat Pipe Theory and Practice: A Sourcebook*. Washington, DC: Hemisphere Publishing Corporation, 1976.

[4] A. Faghri. *Heat Pipe Science and Technology*. Washington, DC: Taylor & Francis, 1995.

[5] G. P. Peterson. *An Introduction to Heat Pipes: Modeling, Testing, and Applications*. New York: John Wiley & Sons, 1994.

[6] D. Reay, R. McGlen, dan P. Kew. *Heat Pipes: Theory, Design and Applications*, ed. ke-6. Oxford: Butterworth-Heinemann, 2014.

[7] J. K. Fink dan L. Leibowitz. *Thermodynamic and Transport Properties of Sodium Liquid and Vapor*. ANL/RE-95/2, Argonne National Laboratory, 1995.

[8] B. H. Yan, C. Wang, dan L. Li. "The Technology of Micro Heat Pipe Cooled Reactor: A Review." *Annals of Nuclear Energy*, vol. 135, 106948, 2020. DOI: 10.1016/j.anucene.2019.106948.

[9] E. Wang, T. Ren, dan L. Li. "Review of Reactor Conceptual Design and Thermal Hydraulic Characteristics for Heat Pipe in Nuclear Systems." *Frontiers in Energy Research*, vol. 11, 1264168, 2023. DOI: 10.3389/fenrg.2023.1264168.

[10] C. Wang, H. Sun, S. Tang, W. Tian, S. Qiu, dan G. Su. "Thermal-Hydraulic Analysis of a New Conceptual Heat Pipe Cooled Small Nuclear Reactor System." *Nuclear Engineering and Technology*, vol. 52, no. 1, pp. 19–26, 2020. DOI: 10.1016/j.net.2019.06.021.

[11] K. A. Woloshun, M. A. Merrigan, dan E. D. Best. *HTPIPE: A Steady-State Heat Pipe Analysis Program—A User's Manual*. Los Alamos National Laboratory, 1988.

[12] Z. Zhang, C. Wang, K. Guo, J. Huang, S. Qiu, dan G. Su. "HEART, a Specific Code for Thermal-Electrical Analysis of Heat Pipe Cooled Nuclear Reactor." *International Journal of Thermal Sciences*, vol. 179, 107666, 2022. DOI: 10.1016/j.ijthermalsci.2022.107666.

[13] S. Wahlquist dan kolaborator. "A Critical Review of Heat Pipe Experiments in Nuclear Energy Applications." *Nuclear Science and Engineering*, vol. 197, no. 5, pp. 719–752, 2023.

[14] C. R. Harris et al. “Array Programming with NumPy.” *Nature*, vol. 585, pp. 357–362, 2020. DOI: 10.1038/s41586-020-2649-2.

[15] J. D. Hunter. “Matplotlib: A 2D Graphics Environment.” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. DOI: 10.1109/MCSE.2007.55.

BAB A
LISTING PROGRAM SIMULATOR

Lampiran ini memuat sumber lengkap `src/heatpipe_simulator.py`. Program menggunakan satuan SI dan menghasilkan ulang seluruh data serta gambar yang dibahas pada Bab V.

```
#!/usr/bin/env python3
"""Reduced-order, steady 1-D sodium heat-pipe simulator.

The model couples axial vapor flow, Darcy liquid return through a homogeneous
wick, and capillary pressure. It is intended for transparent thesis case
studies, not design certification or transient/CFD analysis.

Sodium correlations follow Fink and Leibowitz (1995), ANL/RE-95/2.
All internal calculations use SI units.
"""
from __future__ import annotations

import argparse
import csv
import json
import math
import warnings
from dataclasses import asdict, dataclass, replace
from pathlib import Path
from typing import Iterable

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

R_UNIVERSAL = 8.314462618  # J mol-1 K-1
MOLAR_MASS_NA = 22.98976928e-3  # kg mol-1
K_BOLTZMANN = 1.380649e-23  # J K-1
```

```python
@dataclass(frozen=True)
class SodiumProperties:
    """Saturated-sodium properties; recommended thesis range: 800--1400 K."""

    temperature: float

    def __post_init__(self) -> None:
        if not 371.0 <= self.temperature < 2503.7:
            raise ValueError("Temperature must be in [371, 2503.7) K.")
        if not 800.0 <= self.temperature <= 1400.0:
            warnings.warn(
                f"T={self.temperature:.1f} K is outside the thesis validation "
                "target of 800--1400 K; check extrapolation.",
                stacklevel=2,
            )

    @property
    def pressure(self) -> float:
        """Saturation pressure (Pa), Browning--Potter correlation."""
        ln_p_mpa = (
            11.9463 - 12633.73 / self.temperature
            - 0.4672 * math.log(self.temperature)
        )
        return math.exp(ln_p_mpa) * 1.0e6

    @property
    def latent_heat(self) -> float:
        """Specific enthalpy of vaporization (J kg-1)."""
        tau = 1.0 - self.temperature / 2503.7
        h_fg_kj_kg = 393.37 * tau + 4398.6 * tau**0.29302
        return h_fg_kj_kg * 1000.0

    @property
    def liquid_density(self) -> float:
        """Saturated liquid density (kg m-3)."""
        tau = 1.0 - self.temperature / 2503.7
        return 219.0 + 275.32 * tau + 511.58 * math.sqrt(tau)
```

```python
    @property
    def vapor_density(self) -> float:
        """Ideal-gas approximation to saturated vapor density (kg m-3)."""
        return (
            self.pressure * MOLAR_MASS_NA
            / (R_UNIVERSAL * self.temperature)
        )

    @property
    def liquid_viscosity(self) -> float:
        """Dynamic viscosity of liquid sodium (Pa s)."""
        return math.exp(
            -6.4406 - 0.3958 * math.log(self.temperature)
            + 556.835 / self.temperature
        )

    @property
    def vapor_viscosity(self) -> float:
        """Hard-sphere dilute-gas estimate of vapor viscosity (Pa s).

        A 4.0 Angstrom collision diameter is used because the recommended
        liquid-property report does not provide a sodium-vapor viscosity
        correlation. Replace this approximation for high-fidelity studies.
        """
        mass = MOLAR_MASS_NA / 6.02214076e23
        diameter = 4.0e-10
        return (
            5.0 / (16.0 * diameter**2)
            * math.sqrt(mass * K_BOLTZMANN * self.temperature / math.pi)
        )

    @property
    def surface_tension(self) -> float:
        """Liquid surface tension (N m-1), Van der Waals correlation."""
        return (
            240.5e-3 * (1.0 - self.temperature / 2503.7)**1.126
```

```
        )

    @property
    def liquid_conductivity(self) -> float:
        """Liquid thermal conductivity (W m-1 K-1)."""
        t = self.temperature
        return (
            124.67 - 0.11381 * t + 5.5226e-5 * t**2
            - 1.1842e-8 * t**3
        )

    @property
    def specific_gas_constant(self) -> float:
        return R_UNIVERSAL / MOLAR_MASS_NA

    @property
    def sound_speed(self) -> float:
        """Monatomic ideal-gas sound-speed estimate (m s-1)."""
        return math.sqrt((5.0 / 3.0) * self.specific_gas_constant * self.tempera

    def summary(self) -> dict[str, float]:
        return {
            "temperature_K": self.temperature,
            "saturation_pressure_Pa": self.pressure,
            "latent_heat_J_kg": self.latent_heat,
            "liquid_density_kg_m3": self.liquid_density,
            "vapor_density_kg_m3": self.vapor_density,
            "liquid_viscosity_Pa_s": self.liquid_viscosity,
            "vapor_viscosity_Pa_s": self.vapor_viscosity,
            "surface_tension_N_m": self.surface_tension,
            "liquid_conductivity_W_mK": self.liquid_conductivity,
            "sound_speed_m_s": self.sound_speed,
        }


@dataclass(frozen=True)
class HeatPipeGeometry:
```

```
    evaporator_length: float = 0.25
    adiabatic_length: float = 0.15
    condenser_length: float = 0.60
    vapor_diameter: float = 0.012
    wick_thickness: float = 0.001
    pore_radius: float = 75.0e-6
    permeability: float = 5.0e-10
    contact_angle_deg: float = 0.0
    effective_wick_conductivity: float = 10.0
    bubble_radius: float = 5.0e-6

    def __post_init__(self) -> None:
        positive = {
            key: value for key, value in asdict(self).items()
            if key != "contact_angle_deg"
        }
        if any(value <= 0.0 for value in positive.values()):
            raise ValueError("All geometry and wick properties must be positive.
        if not 0.0 <= self.contact_angle_deg < 90.0:
            raise ValueError("Contact angle must be in [0, 90) degrees.")

    @property
    def total_length(self) -> float:
        return self.evaporator_length + self.adiabatic_length + self.condenser_l

    @property
    def effective_length(self) -> float:
        return (
            self.evaporator_length / 2.0 + self.adiabatic_length
            + self.condenser_length / 2.0
        )

    @property
    def vapor_area(self) -> float:
        return math.pi * self.vapor_diameter**2 / 4.0

    @property
```

```python
    def wick_area(self) -> float:
        r_v = self.vapor_diameter / 2.0
        return math.pi * ((r_v + self.wick_thickness)**2 - r_v**2)

    @property
    def capillary_pressure(self) -> float:
        raise AttributeError("Capillary pressure also requires fluid temperature


@dataclass(frozen=True)
class OperatingConditions:
    heat_input: float = 800.0
    sink_temperature: float = 700.0
    condenser_resistance: float = 0.375
    gravity: float = 0.0
    tilt_deg: float = 0.0
    cells: int = 400

    def __post_init__(self) -> None:
        if self.heat_input < 0.0 or self.sink_temperature <= 0.0:
            raise ValueError(
                "Heat input must be nonnegative and sink temperature positive."
            )
        if self.condenser_resistance <= 0.0 or self.gravity < 0.0:
            raise ValueError(
                "Condenser resistance must be positive and gravity nonnegative."
            )
        if self.cells < 20:
            raise ValueError("Use at least 20 axial cells.")

    @property
    def operating_temperature(self) -> float:
        return self.sink_temperature + self.condenser_resistance * self.heat_inp


@dataclass
class SimulationResult:
```

```
    x: np.ndarray
    section: np.ndarray
    axial_heat_flow: np.ndarray
    mass_flow: np.ndarray
    vapor_velocity: np.ndarray
    reynolds: np.ndarray
    mach: np.ndarray
    vapor_pressure_drop: np.ndarray
    liquid_pressure_drop: np.ndarray
    summary: dict[str, float | str | bool]


def capillary_pressure(geometry: HeatPipeGeometry, fluid: SodiumProperties) -> f
    return (
        2.0 * fluid.surface_tension
        * math.cos(math.radians(geometry.contact_angle_deg))
        / geometry.pore_radius
    )


def axial_heat_profile(
    x: np.ndarray, heat_input: float, geometry: HeatPipeGeometry,
) -> np.ndarray:
    """Heat carried by vapor; x=0 is the closed evaporator end."""
    le, la, lc = (
        geometry.evaporator_length,
        geometry.adiabatic_length,
        geometry.condenser_length,
    )
    q = np.empty_like(x)
    evap = x <= le
    adiab = (x > le) & (x <= le + la)
    cond = x > le + la
    q[evap] = heat_input * x[evap] / le
    q[adiab] = heat_input
    q[cond] = heat_input * (1.0 - (x[cond] - le - la) / lc)
    return np.maximum(q, 0.0)
```

```
def _cumulative_trapezoid(y: np.ndarray, x: np.ndarray) -> np.ndarray:
    result = np.zeros_like(y)
    result[1:] = np.cumsum(0.5 * (y[1:] + y[:-1]) * np.diff(x))
    return result


def simulate_at_temperature(
    geometry: HeatPipeGeometry,
    conditions: OperatingConditions,
    temperature: float,
) -> SimulationResult:
    """Solve the steady axial profile at a prescribed saturation temperature."""
    fluid = SodiumProperties(temperature)
    x = np.linspace(0.0, geometry.total_length, conditions.cells + 1)
    q = axial_heat_profile(x, conditions.heat_input, geometry)
    mdot = q / fluid.latent_heat
    velocity = mdot / (fluid.vapor_density * geometry.vapor_area)
    reynolds = (
        fluid.vapor_density * velocity * geometry.vapor_diameter
        / fluid.vapor_viscosity
    )
    mach = velocity / fluid.sound_speed
    friction = np.where(
        reynolds < 2300.0,
        64.0 / np.maximum(reynolds, 1.0e-12),
        0.3164 / np.maximum(reynolds, 1.0e-12)**0.25,
    )
    friction = np.where(reynolds < 1.0e-12, 0.0, friction)
    dpv_dx = (
        friction / geometry.vapor_diameter
        * 0.5 * fluid.vapor_density * velocity**2
    )
    dpl_dx = (
        fluid.liquid_viscosity * mdot
        / (fluid.liquid_density * geometry.permeability * geometry.wick_area)
```

```
)
dpv = _cumulative_trapezoid(dpv_dx, x)
dpl = _cumulative_trapezoid(dpl_dx, x)
dp_gravity = (
    fluid.liquid_density * conditions.gravity * geometry.total_length
    * math.sin(math.radians(conditions.tilt_deg))
)
p_cap = capillary_pressure(geometry, fluid)
margin = p_cap - dpv[-1] - dpl[-1] - dp_gravity

# Engineering screening limits (Faghri/Chi-style reduced-order expressions).
q_sonic = (
    0.474 * geometry.vapor_area * fluid.latent_heat
    * math.sqrt(fluid.vapor_density * fluid.pressure)
)
q_entrainment = (
    geometry.vapor_area * fluid.latent_heat
    * math.sqrt(
        fluid.vapor_density * fluid.surface_tension
        / (2.0 * geometry.pore_radius)
    )
)
r_v = geometry.vapor_diameter / 2.0
r_i = r_v + geometry.wick_thickness
nucleation_pressure = max(
    2.0 * fluid.surface_tension / geometry.bubble_radius
    - (dpv[-1] + dpl[-1] + dp_gravity),
    0.0,
)
q_boiling = (
    2.0 * math.pi * geometry.evaporator_length
    * geometry.effective_wick_conductivity * temperature
    / (fluid.latent_heat * fluid.vapor_density * math.log(r_i / r_v))
    * nucleation_pressure
)
limits = {
    "capillary": capillary_limit_at_temperature(geometry, conditions, temper
```

```
        "sonic": q_sonic,
        "entrainment": q_entrainment,
        "boiling": q_boiling,
    }
    governing = min(limits, key=limits.get)
    utilization = conditions.heat_input / limits[governing]
    section = np.select(
        [x <= geometry.evaporator_length,
         x <= geometry.evaporator_length + geometry.adiabatic_length],
        ["Evaporator", "Adiabatic"], default="Condenser",
    )
    summary: dict[str, float | str | bool] = {
        "temperature_K": temperature,
        "heat_input_W": conditions.heat_input,
        "max_mass_flow_kg_s": float(np.max(mdot)),
        "max_vapor_velocity_m_s": float(np.max(velocity)),
        "max_reynolds": float(np.max(reynolds)),
        "max_mach": float(np.max(mach)),
        "capillary_pressure_Pa": p_cap,
        "vapor_pressure_drop_Pa": float(dpv[-1]),
        "liquid_pressure_drop_Pa": float(dpl[-1]),
        "gravity_pressure_drop_Pa": dp_gravity,
        "capillary_margin_Pa": float(margin),
        "capillary_limit_W": limits["capillary"],
        "sonic_limit_W": q_sonic,
        "entrainment_limit_W": q_entrainment,
        "boiling_limit_W": q_boiling,
        "governing_limit": governing,
        "limit_utilization": utilization,
        "is_capillary_feasible": bool(margin >= 0.0),
        "is_within_all_screening_limits": bool(utilization <= 1.0 and margin >=
        "is_incompressible_screening_ok": bool(np.max(mach) < 0.3),
        "gravity_m_s2": conditions.gravity,
        "tilt_deg": conditions.tilt_deg,
        "cells": conditions.cells,
    }
    return SimulationResult(
```

```
            x, section, q, mdot, velocity, reynolds, mach, dpv, dpl, summary,
        )


    def simulate(
        geometry: HeatPipeGeometry, conditions: OperatingConditions,
    ) -> SimulationResult:
        return simulate_at_temperature(
            geometry, conditions, conditions.operating_temperature,
        )


    def capillary_limit_at_temperature(
        geometry: HeatPipeGeometry,
        conditions: OperatingConditions,
        temperature: float,
    ) -> float:
        """Find Q for zero capillary margin using monotone bisection."""
        base = replace(conditions, heat_input=1.0)

        def margin(q: float) -> float:
            # Inline the pressure calculation to avoid recursive limit evaluation.
            fluid = SodiumProperties(temperature)
            local = replace(base, heat_input=q)
            x = np.linspace(0.0, geometry.total_length, local.cells + 1)
            heat = axial_heat_profile(x, q, geometry)
            mdot = heat / fluid.latent_heat
            velocity = mdot / (fluid.vapor_density * geometry.vapor_area)
            reynolds = (
                fluid.vapor_density * velocity * geometry.vapor_diameter
                / fluid.vapor_viscosity
            )
            friction = np.where(
                reynolds < 2300.0,
                64.0 / np.maximum(reynolds, 1.0e-12),
                0.3164 / np.maximum(reynolds, 1.0e-12)**0.25,
            )
```

```
            friction = np.where(reynolds < 1.0e-12, 0.0, friction)
            dpv_dx = (
                friction / geometry.vapor_diameter
                * 0.5 * fluid.vapor_density * velocity**2
            )
            dpl_dx = (
                fluid.liquid_viscosity * mdot
                / (fluid.liquid_density * geometry.permeability * geometry.wick_area
            )
            dpv = np.trapezoid(dpv_dx, x)
            dpl = np.trapezoid(dpl_dx, x)
            dpg = (
                fluid.liquid_density * local.gravity * geometry.total_length
                * math.sin(math.radians(local.tilt_deg))
            )
            return capillary_pressure(geometry, fluid) - dpv - dpl - dpg

        if margin(0.0) <= 0.0:
            return 0.0
        low, high = 0.0, 100.0
        for _ in range(30):
            if margin(high) < 0.0:
                break
            high *= 2.0
        else:
            raise RuntimeError("Could not bracket the capillary limit.")
        for _ in range(55):
            mid = 0.5 * (low + high)
            if margin(mid) >= 0.0:
                low = mid
            else:
                high = mid
        return 0.5 * (low + high)


    def operating_limits(
        geometry: HeatPipeGeometry,
```

```
    conditions: OperatingConditions,
    temperatures: Iterable[float],
) -> list[dict[str, float | str]]:
    rows = []
    for temperature in temperatures:
        # A small reference heat input makes boiling a property-based screening
        # value; the other three limits are independent of the imposed load.
        probe = replace(conditions, heat_input=1.0)
        result = simulate_at_temperature(geometry, probe, float(temperature))
        limits = {
            "capillary": float(result.summary["capillary_limit_W"]),
            "sonic": float(result.summary["sonic_limit_W"]),
            "entrainment": float(result.summary["entrainment_limit_W"]),
            "boiling": float(result.summary["boiling_limit_W"]),
        }
        governing = min(limits, key=limits.get)
        rows.append({
            "temperature_K": float(temperature),
            **limits,
            "maximum_transport_W": limits[governing],
            "governing_limit": governing,
        })
    return rows


def write_csv(
    path: Path, rows: list[dict], fieldnames: list[str] | None = None,
) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    if not rows:
        raise ValueError("No rows supplied.")
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=fieldnames or list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)
```

```
def configure_plots() -> None:
    plt.rcParams.update({
        "font.size": 10,
        "axes.grid": True,
        "grid.alpha": 0.25,
        "figure.dpi": 150,
        "savefig.dpi": 300,
        "savefig.bbox": "tight",
    })


def plot_axial(result: SimulationResult, path: Path) -> None:
    configure_plots()
    fig, axes = plt.subplots(3, 1, figsize=(7.0, 8.2), sharex=True)
    axes[0].plot(result.x, result.axial_heat_flow, color="#9b2226")
    axes[0].set_ylabel("Aliran kalor aksial (W)")
    axes[1].plot(result.x, result.vapor_velocity, color="#005f73")
    axes[1].set_ylabel("Kecepatan uap (m/s)")
    axes[2].plot(result.x, result.vapor_pressure_drop, label="Uap", color="#ca67
    axes[2].plot(result.x, result.liquid_pressure_drop, label="Cairan", color="#
    axes[2].set_ylabel("Rugi tekanan kumulatif (Pa)")
    axes[2].set_xlabel("Posisi dari ujung evaporator, x (m)")
    axes[2].legend()
    for ax in axes:
        for boundary in (0.25, 0.40):
            ax.axvline(boundary, color="0.45", linestyle="--", linewidth=0.8)
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)


def plot_limits(rows: list[dict], reference_q: float, path: Path) -> None:
    configure_plots()
    temperature = [row["temperature_K"] for row in rows]
    fig, axes = plt.subplots(2, 1, figsize=(7.2, 6.4), sharex=True)
    styles = {
        "capillary": ("Kapiler", "#9b2226"),
```

```
        "sonic": ("Sonik", "#005f73"),
        "entrainment": ("Entrainment", "#ca6702"),
    }
    for key, (label, color) in styles.items():
        axes[0].plot(temperature, [row[key] for row in rows], label=label, color
    axes[0].axhline(
        reference_q, color="black", linestyle="--", label="Beban kasus acuan",
    )
    axes[0].set_ylabel("Batas kalor (W)")
    axes[0].legend(ncol=2)
    axes[1].plot(temperature, [row["boiling"] for row in rows], color="#0a9396")
    axes[1].set_yscale("log")
    axes[1].set(
        xlabel="Temperatur operasi (K)",
        ylabel="Batas pendidihan (W)",
    )
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)


def plot_schematic(geometry: HeatPipeGeometry, path: Path) -> None:
    """Draw the modeled control volume and counter-current phase paths."""
    configure_plots()
    fig, ax = plt.subplots(figsize=(8.0, 3.2))
    le, la, lc = (
        geometry.evaporator_length, geometry.adiabatic_length,
        geometry.condenser_length,
    )
    length = geometry.total_length
    ax.add_patch(plt.Rectangle((0, 0.25), length, 1.5, fill=False, linewidth=2.0
    ax.add_patch(plt.Rectangle((0, 0.25), length, 0.22, color="#0a9396", alpha=0
    ax.add_patch(plt.Rectangle((0, 1.53), length, 0.22, color="#0a9396", alpha=0
    ax.axvline(le, color="0.35", linestyle="--")
    ax.axvline(le + la, color="0.35", linestyle="--")
    ax.text(le / 2, 2.02, "Evaporator", ha="center", weight="bold")
    ax.text(le + la / 2, 2.02, "Adiabatik", ha="center", weight="bold")
```

```
    ax.text(le + la + lc / 2, 2.02, "Kondenser", ha="center", weight="bold")
    ax.text(le / 2, 2.22, "Q masuk", ha="center", color="#9b2226")
    ax.text(le + la + lc / 2, 2.22, "Q keluar", ha="center", color="#005f73")
    for start in (0.04, 0.22):
        ax.annotate("", xy=(le - 0.02, start + 0.65), xytext=(start, start + 0.6
                    arrowprops=dict(arrowstyle="->", color="#ca6702", lw=1.8))
    ax.annotate("", xy=(le + la + lc - 0.05, 1.0), xytext=(le + 0.02, 1.0),
                arrowprops=dict(arrowstyle="->", color="#ca6702", lw=1.8))
    ax.annotate("", xy=(le + la + 0.05, 0.78), xytext=(length - 0.05, 0.78),
                arrowprops=dict(arrowstyle="->", color="#ca6702", lw=1.8))
    ax.annotate("", xy=(0.05, 0.36), xytext=(length - 0.05, 0.36),
                arrowprops=dict(arrowstyle="->", color="white", lw=1.5))
    ax.annotate("", xy=(0.05, 1.64), xytext=(length - 0.05, 1.64),
                arrowprops=dict(arrowstyle="->", color="white", lw=1.5))
    ax.text(length / 2, 1.18, "Uap jenuh", ha="center", color="#8c510a")
    ax.text(
        length / 2, 0.05,
        "Kapiler: cairan kembali melalui wick (arah berlawanan)",
        ha="center", color="#075b5d",
    )
    ax.text(0, -0.18, "x = 0", ha="center")
    ax.text(length, -0.18, "x = L", ha="center")
    ax.set(xlim=(-0.08, length + 0.08), ylim=(-0.3, 2.45))
    ax.axis("off")
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)


def plot_gravity(rows: list[dict], path: Path) -> None:
    configure_plots()
    labels = [row["label"] for row in rows]
    values = [row["capillary_limit_W"] for row in rows]
    fig, ax = plt.subplots(figsize=(7.2, 4.2))
    bars = ax.bar(
        labels, values,
        color=["#0a9396", "#0a9396", "#ca6702", "#ca6702", "#9b2226"],
```

```
    )
    ax.set_ylabel("Batas kapiler (W)")
    ax.set_xlabel("Percepatan efektif dan orientasi")
    ax.tick_params(axis="x", rotation=20)
    ax.bar_label(bars, fmt="%.0f")
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)


def run_analysis(project_root: Path) -> dict:
    geometry = HeatPipeGeometry()
    conditions = OperatingConditions()
    result = simulate(geometry, conditions)
    data_dir = project_root / "data"
    figure_dir = project_root / "figures"
    data_dir.mkdir(parents=True, exist_ok=True)
    figure_dir.mkdir(parents=True, exist_ok=True)

    axial_rows = []
    for i in range(len(result.x)):
        axial_rows.append({
            "x_m": result.x[i], "section": result.section[i],
            "axial_heat_flow_W": result.axial_heat_flow[i],
            "mass_flow_kg_s": result.mass_flow[i],
            "vapor_velocity_m_s": result.vapor_velocity[i],
            "reynolds": result.reynolds[i], "mach": result.mach[i],
            "vapor_pressure_drop_Pa": result.vapor_pressure_drop[i],
            "liquid_pressure_drop_Pa": result.liquid_pressure_drop[i],
        })
    write_csv(data_dir / "profil_aksial.csv", axial_rows)

    temperatures = np.arange(800.0, 1200.1, 25.0)
    limit_rows = operating_limits(geometry, conditions, temperatures)
    write_csv(data_dir / "batas_operasi.csv", limit_rows)

    load_rows = []
```

```
for heat_input in (300, 400, 600, 800, 1000, 1200, 1400, 1600):
    local = replace(conditions, heat_input=heat_input)
    row = simulate(geometry, local).summary
    load_rows.append(dict(row))
write_csv(data_dir / "studi_beban.csv", load_rows)

gravity_rows = []
for label, gravity, tilt in (
    ("Mikrogravitasi\n0", 0.0, 0.0),
    ("Akselerasi kecil\n0,01; 90°", 0.01, 90.0),
    ("Akselerasi\n0,1; 90°", 0.1, 90.0),
    ("Akselerasi\n1; 90°", 1.0, 90.0),
    ("Bumi\n9,81; 90°", 9.81, 90.0),
):
    local = replace(conditions, gravity=gravity, tilt_deg=tilt)
    q_cap = capillary_limit_at_temperature(geometry, local, 1000.0)
    gravity_rows.append({
        "label": label, "gravity_m_s2": gravity, "tilt_deg": tilt,
        "capillary_limit_W": q_cap,
    })
write_csv(data_dir / "studi_gravitasi.csv", gravity_rows)

mesh_rows = []
reference_dp = None
for cells in (50, 100, 200, 400, 800, 1600):
    local = replace(conditions, cells=cells)
    summary = simulate(geometry, local).summary
    total_dp = (
        float(summary["vapor_pressure_drop_Pa"])
        + float(summary["liquid_pressure_drop_Pa"])
    )
    if cells == 1600:
        reference_dp = total_dp
    mesh_rows.append({"cells": cells, "total_pressure_drop_Pa": total_dp})
for row in mesh_rows:
    row["relative_difference_percent"] = (
        abs(row["total_pressure_drop_Pa"] - reference_dp)
```

```
            / reference_dp * 100.0
        )
    write_csv(data_dir / "konvergensi_grid.csv", mesh_rows)

    pore_rows = []
    for pore_um in (25, 50, 75, 100, 150):
        local_geometry = replace(geometry, pore_radius=pore_um * 1.0e-6)
        pore_rows.append({
            "pore_radius_um": pore_um,
            "capillary_limit_W": capillary_limit_at_temperature(
                local_geometry, conditions, 1000.0,
            ),
            "entrainment_limit_W": simulate_at_temperature(
                local_geometry, replace(conditions, heat_input=1.0), 1000.0,
            ).summary["entrainment_limit_W"],
        })
    write_csv(data_dir / "studi_pori.csv", pore_rows)

    plot_schematic(geometry, figure_dir / "skema-model.png")
    plot_axial(result, figure_dir / "profil-aksial.png")
    plot_limits(limit_rows, conditions.heat_input, figure_dir / "batas-operasi.p
    plot_gravity(gravity_rows, figure_dir / "pengaruh-gravitasi.png")

    output = {
        "model": "Steady 1-D vapor/Darcy-wick capillary model",
        "geometry": asdict(geometry),
        "conditions": asdict(conditions),
        "fluid": SodiumProperties(conditions.operating_temperature).summary(),
        "results": result.summary,
        "derived_geometry": {
            "total_length_m": geometry.total_length,
            "effective_length_m": geometry.effective_length,
            "vapor_area_m2": geometry.vapor_area,
            "wick_area_m2": geometry.wick_area,
        },
    }
    with (data_dir / "ringkasan_kasus.json").open("w", encoding="utf-8") as hand
```

```
        json.dump(output, handle, indent=2, ensure_ascii=False)
    return output


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "--output-dir", type=Path, default=Path(__file__).resolve().parents[1],
        help="Project directory receiving data/ and figures/ (default: project r
    )
    args = parser.parse_args()
    output = run_analysis(args.output_dir.resolve())
    print(json.dumps(output["results"], indent=2, ensure_ascii=False))


if __name__ == "__main__":
    main()
```

BAB B
PENGUJIAN DAN PANDUAN PENGGUNAAN

B.1. Listing Pengujian

Berkas `tests/test_simulator.py` berisi sepuluh pengujian berbasis unittest.

```
"""Regression and physical-consistency tests for the heat-pipe model."""
from __future__ import annotations

import math
import sys
import unittest
from dataclasses import replace
from pathlib import Path

import numpy as np

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

from heatpipe_simulator import (  # noqa: E402
    HeatPipeGeometry,
    OperatingConditions,
    SodiumProperties,
    axial_heat_profile,
    capillary_limit_at_temperature,
    simulate,
)


class TestSodiumProperties(unittest.TestCase):
    def test_normal_boiling_point(self) -> None:
        # Recommended correlation gives approximately 1 atm at 1156 K.
        pressure = SodiumProperties(1156.0).pressure
        self.assertAlmostEqual(pressure / 101325.0, 1.0, delta=0.03)

    def test_property_values_at_1000_k(self) -> None:
        fluid = SodiumProperties(1000.0)
```

```
            self.assertAlmostEqual(fluid.surface_tension, 0.1354, delta=0.0002)
            self.assertAlmostEqual(fluid.liquid_viscosity, 1.81e-4, delta=2.0e-6)
            self.assertGreater(fluid.latent_heat, 3.8e6)
            self.assertLess(fluid.latent_heat, 4.3e6)
            self.assertGreater(fluid.liquid_density, 750.0)
            self.assertLess(fluid.liquid_density, 810.0)

        def test_positive_properties(self) -> None:
            fluid = SodiumProperties(1000.0)
            for value in fluid.summary().values():
                self.assertTrue(math.isfinite(value) and value > 0.0)


    class TestHeatPipeModel(unittest.TestCase):
        def setUp(self) -> None:
            self.geometry = HeatPipeGeometry()
            self.conditions = OperatingConditions()

        def test_axial_energy_profile(self) -> None:
            x = np.linspace(0.0, self.geometry.total_length, 1001)
            heat = axial_heat_profile(x, 800.0, self.geometry)
            self.assertAlmostEqual(heat[0], 0.0)
            self.assertAlmostEqual(heat[-1], 0.0)
            self.assertAlmostEqual(float(np.max(heat)), 800.0)
            # Heat is generated only in the evaporator and removed in the condenser.
            derivative = np.diff(heat) / np.diff(x)
            self.assertGreater(float(np.max(derivative[:250])), 0.0)
            self.assertLess(float(np.min(derivative[-600:])), 0.0)

        def test_microgravity_removes_hydrostatic_term(self) -> None:
            result = simulate(self.geometry, self.conditions)
            self.assertEqual(result.summary["gravity_pressure_drop_Pa"], 0.0)
            self.assertTrue(result.summary["is_capillary_feasible"])

        def test_reference_case_is_capillary_limited(self) -> None:
            result = simulate(self.geometry, self.conditions)
            self.assertEqual(result.summary["governing_limit"], "capillary")
```

```
        self.assertLess(float(result.summary["limit_utilization"]), 1.0)
        self.assertLess(float(result.summary["max_mach"]), 0.3)

    def test_capillary_limit_has_zero_margin(self) -> None:
        q_cap = capillary_limit_at_temperature(self.geometry, self.conditions, 1
        local = replace(
            self.conditions,
            heat_input=q_cap,
            sink_temperature=1000.0 - self.conditions.condenser_resistance * q_c
        )
        result = simulate(self.geometry, local)
        self.assertAlmostEqual(
            float(result.summary["temperature_K"]), 1000.0, delta=1e-8,
        )
        self.assertAlmostEqual(
            float(result.summary["capillary_margin_Pa"]) / 3612.0,
            0.0, delta=1e-6,
        )

    def test_adverse_gravity_reduces_capacity(self) -> None:
        q_micro = capillary_limit_at_temperature(self.geometry, self.conditions,
        adverse = replace(self.conditions, gravity=1.0, tilt_deg=90.0)
        q_adverse = capillary_limit_at_temperature(self.geometry, adverse, 1000.
        self.assertLess(q_adverse, q_micro)
        earth = replace(self.conditions, gravity=9.81, tilt_deg=90.0)
        self.assertEqual(
            capillary_limit_at_temperature(self.geometry, earth, 1000.0), 0.0,
        )

    def test_grid_convergence(self) -> None:
        coarse = simulate(self.geometry, replace(self.conditions, cells=100))
        fine = simulate(self.geometry, replace(self.conditions, cells=1600))
        dp_coarse = (
            float(coarse.summary["vapor_pressure_drop_Pa"])
            + float(coarse.summary["liquid_pressure_drop_Pa"])
        )
        dp_fine = (
```

```
            float(fine.summary["vapor_pressure_drop_Pa"])
            + float(fine.summary["liquid_pressure_drop_Pa"])
        )
        self.assertLess(abs(dp_coarse - dp_fine) / dp_fine, 0.005)

    def test_invalid_geometry_rejected(self) -> None:
        with self.assertRaises(ValueError):
            HeatPipeGeometry(permeability=0.0)


if __name__ == "__main__":
    unittest.main()
```

B.2. Menjalankan Program

Dari direktori proyek, gunakan perintah berikut.

```
python -m pip install -r requirements.txt
python src/heatpipe_simulator.py
python -m unittest discover -s tests -v
```

Keluaran konsol merangkum kasus acuan dalam JSON. Berkas ringkasan kasus pada direktori data menyimpan masukan, sifat fluida, geometri turunan, dan hasil. Berkas CSV lain menyimpan profil aksial serta studi parameter. Jika parameter pada dataclass diubah, seluruh data dan gambar harus dihasilkan ulang sebelum angka dalam naskah diperbarui.

B.3. Interpretasi Status Program

`is_capillary_feasible` Margin kapiler tidak negatif.

`is_within_all_screening_limits` Daya berada di bawah batas pengatur dan kapiler layak menurut model.

`is_incompressible_screening_ok` Mach maksimum lebih kecil dari 0,3.

`governing_limit` Batas yang bernilai minimum pada kasus tersebut.

`limit_utilization` Rasio daya terhadap batas pengatur.

Status bernilai benar tidak berarti desain telah tervalidasi. Pengguna harus memeriksa rentang sifat, asumsi geometri, kondisi batas, dan ketidakpastian korelasi.